



Platform ▾

Solutions ▾

Pricing

Enterprise

Log in

TRY NOW →

All Interview Questions

Angular Interview Questions and Answers

Find 100+ Angular interview questions and answers to assess candidates' skills in components, services, directives, routing, and reactive programming with Angular.

By WeCP Team

As a leading **front-end framework for enterprise-grade web applications**, Angular empowers developers to build **robust, scalable, and maintainable SPAs (Single Page Applications)**. Recruiters must identify professionals skilled in **TypeScript, component architecture, and Angular's reactive patterns**, ensuring delivery of performant and maintainable codebases.

This resource, "**100+ Angular Interview Questions and Answers**," is tailored for recruiters to simplify the evaluation process. It covers topics from **Angular basics to advanced concepts like RxJS, state management, and performance optimization**, including **dependency injection, routing, forms, and module architecture**.



WeCP

WeCP

Team

Team

@WeCP

WeCP is a
leading
talent
assessment
platform that
helps

Whether hiring for **Front-End Developers, Angular Developers, or Full-Stack Engineers**, this guide enables you to assess a candidate's:

- **Core Angular Knowledge:** Understanding of **modules, components, templates, services, dependency injection, and lifecycle hooks**.
- **Advanced Skills:** Expertise in **RxJS for reactive programming, Angular forms (template-driven and reactive), routing and lazy loading, and performance optimization techniques (ChangeDetectionStrategy, OnPush, trackBy)**.
- **Real-World Proficiency:** Ability to **design scalable app architectures, integrate REST APIs, implement state management with NgRx or services, write unit and E2E tests (using Jasmine, Karma, or Protractor/Cypress), and follow Angular best practices**.

For a **streamlined assessment process**, consider platforms like **WeCP**, which allow you to:

- ✅ **Create customized Angular assessments** aligned to role requirements and experience levels.
- ✅ **Include hands-on coding tasks**, such as building components, services, or reactive forms within a simulated IDE.
- ✅ **Proctor assessments remotely** with AI-based integrity monitoring.
- ✅ **Leverage AI-powered grading** to evaluate code quality, logic, and adherence to Angular's architectural standards.

Save time, improve hiring precision, and confidently recruit **Angular professionals** who can build **scalable, high-performance applications from day one**.

Angular Interview Questions

Beginner (40 Questions)

1. What is Angular?
2. What are the key features of Angular?
3. Explain the difference between Angular and AngularJS.
4. What is a component in Angular?
5. What is a module in Angular?
6. What is TypeScript? Why is it used in Angular?
7. What is data binding in Angular?
8. Explain the different types of data binding in Angular.

companies
streamline
their
recruitment
and L&D
process by
evaluating
candidates'
skills through
tailored
assessments



9. What are directives in Angular?
10. What is the purpose of `ngOnInit()` in Angular?
11. What is a template in Angular?
12. What is a service in Angular?
13. How do you create a new Angular component?
14. How do you use the `ngIf` directive?
15. What is the purpose of the `ngFor` directive?
16. What is the Angular CLI?
17. How do you create a new Angular project using the Angular CLI?
18. What is routing in Angular?
19. What is the difference between `ngModel` and `ngModelGroup`?
20. Explain the lifecycle hooks in Angular.
21. What is dependency injection in Angular?
22. What is an observable in Angular?
23. How do you handle HTTP requests in Angular?
24. What is the purpose of the `HttpClientModule`?
25. How do you create a service in Angular?
26. What is the purpose of `@Input` and `@Output` decorators?
27. What is Angular's change detection mechanism?
28. What is the purpose of `ngContent`?
29. What is an Angular Pipe?
30. What is the `async` pipe in Angular?
31. How do you bind events in Angular?
32. What is the difference between a component and a directive in Angular?
33. How do you perform validation in Angular forms?
34. Explain the difference between template-driven and reactive forms in Angular.
35. What are Angular modules and why are they important?
36. What is lazy loading in Angular?
37. What is a subscription in Angular and how do you unsubscribe?
38. How does Angular handle cross-site scripting (XSS)?
39. What is the purpose of the `@angular/forms` module?

40. What is the ng serve command in Angular?

Intermediate (40 Questions)

1. Explain the difference between `ngOnInit()` and `constructor()` in Angular.
2. What are the various lifecycle hooks in Angular, and how are they used?
3. What is an Angular decorator? Name a few commonly used decorators.
4. What is a factory provider in Angular?
5. What is Angular's Change Detection Strategy?
6. What is `ngZone` in Angular, and why is it important?
7. Explain the use of `@ViewChild` and `@ContentChild`.
8. How can you optimize the performance of an Angular application?
9. What is the `trackBy` function in `ngFor` and why is it important?
10. What are Angular interceptors, and how are they used?
11. Explain the concept of Observables and Promises in Angular.
12. What is the purpose of the RxJS library in Angular?
13. How does Angular handle forms validation with custom validators?
14. What is the difference between `ngOnInit()` and `ngAfterViewInit()`?
15. What are lazy-loaded modules, and how do they work in Angular?
16. What is `ngModule` in Angular, and how does it help with modularity?
17. How do you pass data between Angular components?
18. What are Angular Guards, and what are their different types?
19. How can you handle errors in Angular HTTP requests?
20. What is the role of the `HttpClientModule` and how do you use it?
21. Explain the difference between `HttpClient` and `Http` in Angular.
22. What is the role of the Injector in Angular?
23. How does Angular handle routing?
24. What is an Angular resolver, and when would you use it?
25. What is `ngModel` used for in Angular forms?
26. How do you create a custom pipe in Angular?
27. What are Angular modules, and how do they help in application structure?

28. What is the difference between `ngOnChanges()` and `ngDoCheck()` lifecycle hooks?
29. Explain how the Angular module system works with imports, exports, and providers.
30. How do you implement global error handling in Angular applications?
31. How can you optimize large Angular applications?
32. What is the difference between `ngFor` and `ngForOf`?
33. How do you optimize Angular change detection using `ChangeDetectionStrategy`?
34. What are some performance best practices when working with Angular?
35. What is an async pipe, and how is it used in Angular templates?
36. What is the difference between `HttpClient` and `HttpModule` in Angular?
37. What is the role of the `ngFor` directive in Angular?
38. How can you dynamically load a component in Angular?
39. Explain the concept of `ngOnDestroy()` and when to use it.
40. What are some common issues with routing in Angular and how can you resolve them?

Experienced (40 Questions)

1. Explain the concept of Angular's Ahead-of-Time (AOT) compilation.
2. What is Angular Universal, and why is it used?
3. How can you improve the performance of an Angular application?
4. What are the key differences between Reactive Forms and Template-Driven Forms in Angular?
5. Explain how Angular's Dependency Injection works in depth.
6. How would you configure lazy loading for feature modules in Angular?
7. What are the advantages of using Angular CLI in a large application?
8. What are Angular decorators and how do they help in application development?
9. How does Angular handle memory leaks, and what are some strategies to avoid them?
10. Explain the use of `ngOnInit()` and `ngAfterViewInit()` lifecycle hooks with examples.

11. How do you secure an Angular application from cross-site scripting (XSS) attacks?
12. What are the pros and cons of using Angular vs. React or Vue.js?
13. Explain the concept of modules in Angular. What are the different types of modules in Angular?
14. How do you handle state management in large Angular applications?
15. What is the role of NgModules in Angular, and how do they contribute to application structure?
16. Explain how to create a custom Angular directive and provide an example use case.
17. What is the purpose of ngZone in Angular, and how do you work with it?
18. What is server-side rendering (SSR) in Angular?
19. How can you implement authentication and authorization in Angular?
20. What is the role of HttpInterceptor in handling HTTP requests?
21. Explain the difference between ChangeDetectionStrategy.Default and ChangeDetectionStrategy.OnPush.
22. How do you optimize the bundle size in Angular applications?
23. What is the Angular Service Worker, and how does it support Progressive Web Apps (PWAs)?
24. What is the role of @ViewChild and @ContentChild in Angular?
25. Explain the difference between component state and application state in Angular.
26. What are Angular zones, and how do they work to detect changes?
27. What are Observables, and how are they different from Promises in Angular?
28. How would you approach debugging and troubleshooting performance issues in Angular applications?
29. How do you handle API responses using async/await in Angular?
30. What are the various strategies for testing Angular components?
31. How would you create a custom Angular pipe, and when should you use them?
32. What is an Angular resolver, and how do you use it for pre-fetching data in routing?
33. How do you implement internationalization (i18n) in Angular?
34. Explain how the Angular Router works with lazy-loaded modules.

35. What are the main differences between Angular's HttpClient and Angular's HttpModule?
36. How would you optimize change detection in a large Angular application?
37. What is the role of the Renderer2 service in Angular?
38. Explain how to implement dynamic form generation in Angular.
39. How would you manage forms with complex validation logic in Angular?
40. How can you integrate Angular with third-party libraries and services (like jQuery or D3.js)?

Angular Interview Questions and Answers

Beginners (Q&A)

1. What is Angular?

Angular is a **platform and framework** for building single-page client applications using **HTML, CSS, and TypeScript**. Developed and maintained by **Google**, Angular is a **component-based** framework that uses **dependency injection** to create modular, reusable, and maintainable applications.

Angular provides a set of tools and libraries to facilitate the development of complex web applications, including **routing** (to manage navigation), **forms** (template-driven and reactive forms), **HTTP client** (to interact with RESTful APIs), **pipes** (to transform data in templates), and **directives** (to modify the DOM).

The framework follows the **MVC** (Model-View-Controller) design pattern, but Angular's architecture is more focused on **components**, which act as the building blocks of an application.

With **two-way data binding**, **lazy loading**, and **strong support for unit testing**, Angular is a powerful choice for developing modern web apps with rich user interfaces. It is typically used for building **enterprise-grade applications**, **progressive web apps (PWAs)**, and complex dynamic interfaces.

2. What are the key features of Angular?

Angular has several key features that make it a robust framework for building modern web applications:

1. **Component-Based Architecture:** Angular applications are built using components, which encapsulate the view (UI), logic, and data for a particular part of the user interface. Components are reusable and can be composed to create complex UIs.
2. **Two-Way Data Binding:** Angular facilitates two-way data binding, where changes in the model automatically propagate to the view and vice versa, simplifying the synchronization of the UI with the application data.
3. **Dependency Injection (DI):** Angular's DI mechanism helps in managing the dependencies between different parts of the application, making it easier to inject services, directives, and other dependencies into components and other parts of the app.
4. **Routing:** Angular's router allows developers to navigate between different views or components, manage navigation state, and handle complex routing scenarios, such as lazy loading of modules.
5. **RxJS and Observables:** Angular uses RxJS for reactive programming, which allows handling asynchronous events and managing streams of data more effectively. Observables are a key part of the Angular ecosystem, particularly in dealing with HTTP requests.
6. **Directives:** Directives in Angular are used to manipulate the DOM, create custom HTML tags, or bind data to existing HTML elements. Angular provides built-in directives like ngFor and ngIf.
7. **Pipes:** Pipes are used for transforming data in templates. They provide a declarative way of formatting data (e.g., for currency, dates, or text transformations).
8. **Forms Management:** Angular supports both template-driven and reactive forms, which provide powerful features for handling complex form validations, custom validations, and form control management.
9. **Modularization:** Angular applications are structured into modules that allow for better organization, lazy loading, and optimized performance. Modules group related components, services, and other code.
10. **Ahead-of-Time (AOT) Compilation:** Angular compiles templates and components during build time rather than at runtime, resulting in faster rendering and improved application performance.

3. Explain the difference between Angular and AngularJS.

Angular and AngularJS refer to two different versions of the framework, with **AngularJS** being the original version (1.x), while **Angular** (starting from version 2.x) is a complete

rewrite of AngularJS. The primary differences between AngularJS and Angular are:

1. **Language:**

- **AngularJS** uses **JavaScript** as the programming language.
- **Angular** uses **TypeScript**, a superset of JavaScript, which provides better tooling, static typing, and object-oriented features.

2. **Architecture:**

- **AngularJS** follows the **MVC (Model-View-Controller)** pattern.
- **Angular** uses a **component-based architecture**, which is more modular and maintainable.

3. **Performance:**

- **AngularJS** uses **two-way data binding** and **digest cycles**, which can lead to performance issues in complex applications.
- **Angular** improves performance with **change detection strategies**, **AOT (Ahead-of-Time) Compilation**, and **tree shaking**.

4. **Directives:**

- In **AngularJS**, directives were a fundamental part of the framework, often requiring verbose syntax.
- In **Angular**, directives are still present but are used more concisely, with a more flexible system for building components.

5. **Mobile Support:**

- **AngularJS** was not optimized for mobile performance.
- **Angular** was designed with mobile-first principles, offering better performance and responsiveness on mobile devices.

6. **Dependency Injection:**

- **AngularJS** has a simpler dependency injection mechanism.
- **Angular** uses a more powerful and flexible **dependency injection (DI)** system that allows for more complex scenarios and better modularization.

7. **Routing:**

- **AngularJS** routing was less robust, with no native support for lazy loading.
- **Angular** has a powerful, highly configurable **router** with support for **lazy loading**, **guards**, and **preloading strategies**.

4. What is a component in Angular?

A **component** in Angular is a fundamental building block of the application. It is used to define the structure, behavior, and styling of a part of the user interface (UI). Each component in Angular is made up of three key parts:

1. **HTML Template:** Defines the view (the layout, structure, and UI elements) that the component will render.
2. **CSS Styles:** Defines the styles specific to the component. These can either be inline styles or external style files scoped to the component.
3. **TypeScript Class:** Contains the logic for the component. This includes properties, methods, lifecycle hooks, and any event handlers that control how the component behaves.

Every Angular component is associated with a **selector**, which is the HTML tag that references the component in the template. A component can also have **input and output bindings** to communicate with other components.

Example of a simple Angular component:

```
@Component({
  selector: 'app-my-component',
  templateUrl: './my-component.component.html',
  styleUrls: ['./my-component.component.css']
})
export class MyComponent {
  title: string = 'Hello, Angular!';

  changeTitle(newTitle: string) {
    this.title = newTitle;
  }
}
```

5. What is a module in Angular?

In Angular, a **module** is a logical unit of code that groups related components, directives, pipes, and services. Angular modules help in organizing an application into cohesive blocks of functionality and improve maintainability.

An **Angular module** is defined using the `@NgModule` decorator. The most important properties of an Angular module are:

- **declarations:** Declares the components, directives, and pipes that belong to the module.
- **imports:** Lists other modules whose exported components, directives, and pipes are needed by this module.
- **providers:** Configures the services that can be injected into components and other parts of the module.
- **bootstrap:** Specifies the root component that Angular should use to start the application (typically the AppComponent).

Example:

```
@NgModule({  
  declarations: [AppComponent, OtherComponent],  
  imports: [BrowserModule],  
  providers: [AppService],  
  bootstrap: [AppComponent]  
})  
export class AppModule {}
```

Modules help in organizing large applications, enabling features like **lazy loading**, **modularization**, and **scoping** of services.

6. What is TypeScript? Why is it used in Angular?

TypeScript is a **superset of JavaScript** developed by Microsoft. It adds **static typing** to JavaScript, meaning that variables and functions can be typed explicitly, allowing for better tooling, error checking, and code refactoring. TypeScript also supports features like **classes**, **interfaces**, and **generics**, making it more suitable for building large-scale applications.

In Angular, **TypeScript** is used for:

1. **Better Development Experience:** TypeScript's static typing and enhanced tooling (e.g., autocompletion, error detection, and refactoring) make the development process smoother and reduce the chances of bugs.
2. **Object-Oriented Programming:** TypeScript supports modern programming paradigms such as object-oriented programming, making Angular applications more structured, modular, and maintainable.

3. **Compatibility:** TypeScript compiles down to regular JavaScript, meaning it works in any environment that supports JavaScript (browsers, Node.js, etc.).
4. **Angular Ecosystem:** TypeScript is the official language for Angular, and the framework's decorators, such as `@Component`, `@NgModule`, and `@Injectable`, are designed specifically to work with TypeScript.

7. What is data binding in Angular?

Data binding in Angular is a mechanism for synchronizing the data between the **component** (TypeScript class) and the **view** (HTML template). It allows the dynamic updating of the UI when the data changes, as well as capturing user input and updating the component's properties accordingly.

There are four types of data binding in Angular:

Interpolation (`{{ }}`): It is used to bind component data to the view (e.g., display a property value).

```
<h1>{{ title }}</h1>
```

Property Binding (`[ ]`): It binds a property of an HTML element to a component property.

```
<img [src]="imageUrl">
```

Event Binding (`(( ))`): It binds an event (such as click, keyup) from the view to the component.

```
<button (click)="onClick()">Click me</button>
```

Two-Way Binding (`[( )]`): It binds both a property and an event. Changes in the input field will update the component's data, and changes in the component's data will reflect in the input field.

```
<input [(ngModel)]="username">
```

8. Explain the different types of data binding in Angular.

The main types of data binding in Angular are:

Interpolation: Allows you to embed component properties into HTML. It's mostly used for **displaying values** in the template.

```
<p>{{ componentValue }}</p>
```

Property Binding: Binds an element property to a component property. It helps in **dynamically setting HTML element properties**.

```
<img [src]="imageUrl" alt="Dynamic image">
```

Event Binding: Allows the view to listen to events like click, keyup, etc., and call methods in the component in response.

```
<button (click)="handleClick()">Click me</button>
```

Two-Way Binding: This is a combination of property binding and event binding. The **ngModel** directive is used to bind the data in both directions (model to view and view to model).

```
<input [(ngModel)]="username">
```

9. What are directives in Angular?

Directives in Angular are special markers in the DOM that tell Angular to attach a behavior to an element, component, or another directive. They can be used to modify the DOM structure, style elements, or create reusable components.

There are three types of directives in Angular:

1. **Component Directives:** These are the most common type of directive and represent the **component itself**. They encapsulate the view and logic for a part of the user interface.

Structural Directives: These change the structure of the DOM by adding or removing elements. Common examples include `*ngFor` (looping through collections) and `*ngIf` (conditionally adding/removing elements).

```
<div *ngIf="showMessage">This is a message!</div>
```

Attribute Directives: These change the appearance or behavior of an element. Examples include `ngClass`, `ngStyle`, or custom directives that manipulate the DOM.

```
<div [ngClass]="{ 'active': isActive }">Styled Content</div>
```

10. What is the purpose of ngOnInit() in Angular?

ngOnInit() is a **lifecycle hook** in Angular, which is called **once** after the component's data-bound properties are initialized. It is part of the OnInit interface and is typically used to initialize data or perform tasks that should occur after the component's view has been fully initialized.

Some typical use cases for ngOnInit():

- Fetching data from APIs or services.
- Setting up component properties.
- Initializing any complex logic that depends on bindings.

Example:

```
export class AppComponent implements OnInit {  
  data: any;  
  
  ngOnInit() {  
    this.loadData();  
  }  
  
  loadData() {  
    // Load data from service  
    this.data = ['Item 1', 'Item 2'];  
  }  
}
```

This hook is ideal for handling any initialization logic that depends on the component's **inputs** and must be done **after** Angular has set those inputs.

11. What is a template in Angular?

In Angular, a **template** is the **view layer** of an Angular component, where you define the structure and layout of the user interface. It is written using **HTML** along with Angular's templating syntax to add dynamic behavior, data binding, event handling, and conditional rendering. Templates allow developers to express the UI declaratively, meaning you describe what you want to render, rather than how to render it.

A template consists of:

- **HTML Markup:** Standard HTML that represents the structure of the UI.
- **Bindings:** Angular templates can bind data from the component's class to the HTML elements. This includes property binding, event binding, and two-way binding.
 - **Property Binding:** Allows binding of component properties to HTML element properties. E.g., `[src]="imageUrl"` binds the component's `imageUrl` to the `src` attribute of an `img` tag.
 - **Event Binding:** Allows binding of component methods to DOM events. E.g., `(click)="onClick()"` binds the click event to the `onClick()` method in the component.
 - **Two-way Binding:** A combination of property binding and event binding. It's commonly used with `ngModel` to create a two-way data flow between the component and the template.
 - **Example:** `<input [(ngModel)]="username">` binds the `username` property both to the input field's value and to any changes made by the user.
- **Directives:** Angular templates use **directives** to modify the DOM behavior or appearance. Directives are special markers in the DOM that tell Angular to attach behavior to an element or manipulate the DOM structure.
 - **Structural Directives:** These change the structure of the DOM. Examples are `*ngIf` (conditionally add/remove elements) and `*ngFor` (render a list of elements).
 - **Attribute Directives:** These modify the appearance or behavior of a DOM element without changing the DOM structure. Examples include `ngClass`, `ngStyle`, and custom directives.
- **Pipes:** Angular templates also use **pipes** to transform data for display. Pipes are typically used to format data, such as displaying dates or numbers, or filtering a list.
 - Example: `{{ date | date:'shortDate' }}` to format a date.

Example of an Angular component with a template:

```
@Component({
  selector: 'app-example',
  template: `
    <h1>{{ title }}</h1>
```

```

    <button (click)="changeTitle()">Change Title</button>
  ,
  })
  export class ExampleComponent {
    title = 'Hello Angular';

    changeTitle() {
      this.title = 'Title Changed';
    }
  }

```

In the template above, `{{ title }}` binds the component's title property to the view, and the `(click)` binding calls the `changeTitle()` method on button click.

12. What is a service in Angular?

In Angular, a **service** is a class that typically contains **logic** for performing tasks that are not directly related to user interface rendering, such as handling data operations, business logic, and interaction with external APIs or other backend services. Services help to **separate concerns**, promoting the reuse of logic and keeping your components **lean** and focused on managing the view and user interaction.

Key characteristics of Angular services:

- **Business Logic:** Services contain business logic such as validating data, making API calls, managing state, and more. They help reduce redundancy by centralizing complex logic in one place.
- **Dependency Injection:** Angular provides a powerful **dependency injection** (DI) system, which is used to **inject services** into components, directives, and other services. This helps manage service lifecycles and ensures that the service is available throughout the application.
 - Example: A service can be injected into a component using the constructor.
- **Singleton:** By default, services in Angular are **singleton objects**. This means that the same instance of a service is used throughout the application, making it ideal for shared state management or caching data.
- **Separation of Concerns:** A service should focus on business logic, leaving UI-related operations to the components. This makes the codebase easier to test, maintain, and scale.

Example of a simple service in Angular:

```
@Injectable({
  providedIn: 'root'
})
export class UserService {
  private apiUrl = 'https://api.example.com/users';

  constructor(private http: HttpClient) {}

  getUserData() {
    return this.http.get(this.apiUrl);
  }
}
```

In this example, `UserService` is a service that makes an HTTP request to get user data from an API. The `HttpClient` service is injected into the service constructor, enabling it to fetch data from a remote server.

In Angular, services are often provided using the `@Injectable()` decorator, and the `providedIn: 'root'` metadata ensures the service is available globally in the application.

13. How do you create a new Angular component?

To create a new component in Angular, the **Angular CLI (Command Line Interface)** provides a simple command to generate the component and all necessary files, including the TypeScript class, HTML template, and CSS styles.

Here's how you can create a component:

1. Using Angular CLI:

- Open a terminal and navigate to your Angular project directory.

Run the following command to create a new component:

```
ng generate component component-name
```

Alternatively, you can use the shorthand version:

```
ng g c component-name
```

This command will generate four files for the component:

- **component-name.component.ts**: The TypeScript file containing the component's logic.
- **component-name.component.html**: The template (view) for the component.
- **component-name.component.css**: The component's styles.
- **component-name.component.spec.ts**: The testing file (for unit tests).

Example: Suppose we want to create a component named `user-profile`. Run the following command:

```
ng g c user-profile
```

2. This will generate:

- `user-profile.component.ts`
- `user-profile.component.html`
- `user-profile.component.css`
- `user-profile.component.spec.ts`

Manual Creation: If you prefer to manually create a component, you'll need to create the `.ts`, `.html`, and `.css` files, and use the `@Component` decorator to define the component. Example:

```
@Component({
  selector: 'app-user-profile',
  templateUrl: './user-profile.component.html',
  styleUrls: ['./user-profile.component.css']
})
export class UserProfileComponent {
  user = { name: 'John Doe', age: 30 };
}
```

The **selector** specifies the tag that will be used to embed this component in the HTML template, and the **templateUrl** and **styleUrls** link to the component's HTML and CSS files.

14. How do you use the `ngIf` directive?

The `*ngIf` directive is a **structural directive** in Angular that conditionally includes or excludes a part of the DOM based on a given expression. If the expression evaluates to true, the element is added to the DOM; if it evaluates to false, the element is removed.

Syntax:

```
<div *ngIf="condition">This is visible if condition is true</div>
```

- **condition:** This is a Boolean expression that determines whether the content should be displayed. If the condition is true, the element is rendered; if false, it is removed from the DOM.
- Example:

```
<div *ngIf="isVisible">This content will only appear if 'isVisible'  
is true.</div>
```

In the corresponding TypeScript file:

```
export class MyComponent {  
  isVisible = true;  
}
```

ngIf with else:

You can also use the else keyword to specify an alternative template when the condition is false.

```
<div *ngIf="isVisible; else noContent">Content is visible.</div>  
<ng-template #noContent><div>Content is not visible.</div></ng-  
template>
```

In this case, if isVisible is false, the noContent template will be displayed instead.

15. What is the purpose of the ngFor directive?

The *ngFor directive is another **structural directive** in Angular, used for **iterating over a collection (like an array or list)** and rendering an element for each item in the collection. It works similarly to a loop in traditional programming languages.

Syntax:

```
<div *ngFor="let item of items">
  {{ item }}
</div>
```

- **let item:** Declares a local variable (item) for each iteration.
- **of items:** Specifies the iterable collection (items), which could be an array or list.

Example:

```
<ul>
  <li *ngFor="let user of users">
    {{ user.name }}
  </li>
</ul>
```

In the corresponding TypeScript file:

```
export class MyComponent {
  users = [{ name: 'John' }, { name: 'Jane' }];
}
```

The `*ngFor` will create a `<li>` for each object in the `users` array and display the `name` property.

`ngFor` with index:

You can also access the index of the iteration using the index variable.

```
<div *ngFor="let item of items; let i = index">
  {{ i }}: {{ item }}
</div>
```

16. What is the Angular CLI?

The **Angular CLI** (Command Line Interface) is a powerful tool provided by the Angular team to help developers create, manage, build, test, and deploy Angular applications

from the command line. The CLI abstracts many of the tasks that developers would typically need to do manually, such as generating components, services, modules, and routing configurations, as well as setting up build configurations and running tests.

Key features of Angular CLI:

1. **Project Setup:** Quickly generate a new Angular project with all necessary configurations.
 - Command: `ng new my-app`
2. **Generating Components/Services/Modules:** The CLI can automatically generate components, services, modules, etc., with the correct folder structure and boilerplate code.
 - Example: `ng generate component component-name`
3. **Development Server:** The CLI includes a development server that can serve your Angular application and auto-reload the page as you make changes.
 - Command: `ng serve`
4. **Building and Optimization:** Angular CLI handles building the application and can optimize it for production by minifying files, enabling Ahead-of-Time (AOT) compilation, and other optimizations.
 - Command: `ng build --prod`
5. **Testing:** Angular CLI integrates with testing frameworks like Jasmine and Karma to run unit tests and end-to-end (E2E) tests.
 - Command: `ng test` (unit tests) or `ng e2e` (E2E tests)
6. **Deploying:** It can also be used to deploy Angular applications to various environments with minimal setup.

To install Angular CLI:

```
npm install -g @angular/cli
```

17. How do you create a new Angular project using the Angular CLI?

To create a new Angular project using Angular CLI, follow these steps:

Install Angular CLI (if not already installed): You need to install Angular CLI globally on your machine if you don't have it installed.

```
npm install -g @angular/cli
```

Create a New Project: Run the `ng new` command to create a new Angular project. You can specify the name of your project as the first argument.

```
ng new my-angular-app
```

1. This will:
 - Generate a new Angular project.
 - Create a project directory with all the necessary files and folders (such as `src`, `app`, `angular.json`, etc.).
 - Optionally, you can add features such as routing and specify the stylesheet format (CSS, SCSS, etc.).

Navigate to the Project Directory:

```
cd my-angular-app
```

Serve the Application: Run the Angular development server to see the application in the browser:

```
ng serve
```

1. The application will now be accessible at `http://localhost:4200/`.

18. What is routing in Angular?

Routing in Angular allows navigation between different views or pages in a single-page application (SPA). The Angular Router is a powerful tool that enables developers to define routes for different components and handle navigation between them.

Key concepts of Angular routing:

1. **Routes Configuration:** Routes are defined in a **routing module** (typically `app-routing.module.ts`), where you map URL paths to specific components.
2. **RouterOutlet:** In the main template (`app.component.html`), the `<router-outlet>` directive is used to mark where the routed component will be displayed.
3. **Navigation:** The Router enables navigation based on URL changes, whether by using **links** (`routerLink`) or programmatically using the Router service.
4. **Route Guards:** Angular provides route guards (like `CanActivate` and `CanDeactivate`) to protect routes and control access to certain routes based

on conditions.

Example of Routing in Angular:

Defining Routes:In `app-routing.module.ts`:

```
const routes: Routes = [
  { path: 'home', component: HomeComponent },
  { path: 'about', component: AboutComponent },
  { path: '', redirectTo: '/home', pathMatch: 'full' }
];
```

Using RouterOutlet:In `app.component.html`:

```
<router-outlet></router-outlet>
```

Navigating via Links:In the template:

```
<a routerLink="/about">Go to About</a>
```

Navigating Programmatically:In the component:

```
constructor(private router: Router) {}

goToHome() {
  this.router.navigate(['/home']);
}
```

19. What is the difference between ngModel and ngModelGroup?

Both `ngModel` and `ngModelGroup` are part of Angular's **FormsModule** used for working with forms in Angular. They are related but serve different purposes:

- **ngModel:**
 - `ngModel` is used to bind an input field to a specific property in the component. It allows **two-way data binding**, meaning changes in the input field are reflected in the component's property, and changes to the component's property are reflected in the input field.
 - `ngModel` is typically used on **form controls** (such as inputs, selects, checkboxes).

Example:

```
<input [(ngModel)]="username" />
```

- **ngModelGroup:**
 - ngModelGroup is used in **nested forms** or **form groups** to group multiple form controls together. It acts as a container for multiple ngModel directives and helps manage the form as a whole.
 - It's primarily used with **Reactive Forms** to group form controls logically.

Example:

```
<form #form="ngForm">  
  <div ngModelGroup="address">  
    <input ngModel name="street" />  
    <input ngModel name="city" />  
  </div>  
</form>
```

ngModelGroup is typically used when you need to organize a group of form controls under a single name for easier validation and management.

20. Explain the lifecycle hooks in Angular.

Angular components have a set of lifecycle hooks that allow developers to hook into different stages of a component's lifecycle. These hooks allow you to add custom behavior when certain events occur during the component's creation, update, and destruction.

Key lifecycle hooks in Angular:

1. **ngOnInit():**
 - Called **once** after the component's **inputs** have been initialized.
 - Used for **initializing data** or fetching data from services.
2. **ngOnChanges():**
 - Called whenever an **input property** of the component changes.
 - Useful for reacting to changes in input values passed from the parent component.
3. **ngDoCheck():**

- Called during every **change detection cycle**, allowing you to detect changes that Angular might not detect automatically.
- Used for **custom change detection** or deep object comparison.

4. **ngAfterContentInit():**

- Called **once** after Angular has projected content into the component (via `ng-content`).
- Useful when the component has content projected from a parent component.

5. **ngAfterContentChecked():**

- Called after **Angular checks projected content**.
- Useful to respond after the content projection is checked for changes.

6. **ngAfterViewInit():**

- Called **once** after the component's view and its child views have been initialized.
- Useful for performing any **view-related initialization**.

7. **ngAfterViewChecked():**

- Called after Angular checks the component's view and child views.
- Useful for handling any post-view rendering logic.

8. **ngOnDestroy():**

- Called just before Angular destroys the component.
- Useful for cleaning up resources, unsubscribing from observables, or clearing timers.

Each lifecycle hook allows developers to implement logic at a particular point in the component's lifecycle, providing a powerful way to manage the component's behavior.

21. What is dependency injection in Angular?

Dependency Injection (DI) is a design pattern in Angular used to manage how objects or services are created and injected into components or other services. Instead of creating dependencies manually within a class, Angular provides an **inversion of control**, where dependencies (such as services, values, or objects) are injected into a class by an external source, often by the Angular framework itself. This promotes modularity, code reusability, and testability.

How DI Works in Angular:

- **Injectors:** Angular has an injector mechanism that resolves the dependency graph. The injector provides instances of services and other dependencies to components and services that request them.
- **Providers:** A provider is an instruction to the Angular DI system on how to obtain a value for a dependency. Providers are typically declared in the `@Injectable()` decorator or in module-level providers.
- **Decorators:** You use decorators like `@Injectable()` to mark services as available for DI and `@Component()` for components that might need dependencies injected.

Example:

Imagine a `UserService` that depends on `HttpClient` to fetch user data:

```
@Injectable({
  providedIn: 'root'
})
export class UserService {
  constructor(private http: HttpClient) {}

  getUsers() {
    return this.http.get('https://api.example.com/users');
  }
}
```

Now, in your component, you inject `UserService` using Angular's DI:

```
@Component({
  selector: 'app-user-list',
  template: `<ul><li *ngFor="let user of users">{{ user.name }}</li>
</ul>`
})
export class UserListComponent {
  users: any[];

  constructor(private userService: UserService) {}

  ngOnInit() {
```

```
    this.userService.getUsers().subscribe(users => this.users =  
    users);  
  }  
}
```

The UserService instance is automatically injected into the UserListComponent constructor, so you don't have to create an instance manually.

22. What is an observable in Angular?

An **Observable** is a core concept in **RxJS (Reactive Extensions for JavaScript)**, which is used extensively in Angular for handling asynchronous operations like HTTP requests, event handling, and user input.

- An **Observable** represents a stream of data that can be observed over time. You can subscribe to it to get notifications when the data changes or when events occur.
- Observables allow you to handle asynchronous operations more declaratively using operators like map, filter, mergeMap, and switchMap.

Key Points about Observables:

- **Lazy:** Observables are lazy, meaning they don't execute until you subscribe to them.
- **Multiple Values:** Unlike Promises that handle a single value, an Observable can emit multiple values over time (such as streaming data, user input, etc.).
- **Unsubscribing:** It's important to unsubscribe from an Observable when no longer needed to prevent memory leaks. Angular provides automatic unsubscription in some cases (like in async pipe).

Example of using an Observable:

```
import { Observable } from 'rxjs';  
  
let numbers = new Observable(observer => {  
  observer.next(1);  
  observer.next(2);  
  observer.next(3);  
  observer.complete();  
});  
  
numbers.subscribe({
```

```

    next: value => console.log(value), // 1, 2, 3
    complete: () => console.log('Done')
  });

```

In Angular, observables are often used with **HTTP requests** or **event handling**.

23. How do you handle HTTP requests in Angular?

In Angular, HTTP requests are handled using the `HttpClient` module, which provides methods to perform RESTful operations like GET, POST, PUT, DELETE, etc. These requests return **Observables**, which can be subscribed to in components or services.

Steps to Handle HTTP Requests:

1. **Import HttpClientModule:** You need to import the `HttpClientModule` in the application module (`app.module.ts`).

```

import { HttpClientModule } from '@angular/common/http';

@NgModule({
  declarations: [AppComponent],
  imports: [HttpClientModule],
  bootstrap: [AppComponent]
})
export class AppModule {}

```

2. **Inject HttpClient in Service:** Create a service where HTTP requests are made and inject `HttpClient` into the service's constructor.

```

@Injectable({
  providedIn: 'root'
})
export class UserService {
  private apiUrl = 'https://api.example.com/users';

  constructor(private http: HttpClient) {}

  getUsers() {
    return this.http.get(this.apiUrl);
  }
}

```

```

    }

    addUser(user: any) {
      return this.http.post(this.apiUrl, user);
    }
  }
}

```

3. **Subscribe to the Observable:** In your component, call the service method and subscribe to the Observable to get the response.

```

@Component({
  selector: 'app-user-list',
  template: `<ul><li *ngFor="let user of users">{{ user.name }}</li>
</ul>`
})
export class UserListComponent {
  users: any[];

  constructor(private userService: UserService) {}

  ngOnInit() {
    this.userService.getUsers().subscribe(users => this.users =
users);
  }
}

```

4. **Error Handling:** Use the catchError operator to handle errors gracefully.

```

import { catchError } from 'rxjs/operators';
import { of } from 'rxjs';

getUsers() {
  return this.http.get(this.apiUrl).pipe(
    catchError(error => {
      console.error('Error fetching users:', error);
      return of([]); // Return empty array in case of error
    })
  );
}

```

```
);  
}
```

24. What is the purpose of the HttpClientModule?

The **HttpClientModule** is an Angular module that provides an easy-to-use API for making HTTP requests to external services (REST APIs, databases, etc.) in Angular applications. It provides the **HttpClient** service, which is used to perform GET, POST, PUT, DELETE, and other HTTP operations.

Key points about HttpClientModule:

- **HttpClient** is a powerful, flexible, and secure way to interact with APIs.
- It supports Observables, which means responses from HTTP requests are asynchronous and can be handled reactively.
- Automatically transforms JSON responses into JavaScript objects.
- Supports **request/response interception**, which can be used for logging, authorization, and error handling.

To use HttpClient, you need to:

1. Import HttpClientModule in your app.module.ts.
2. Inject the HttpClient service into your components or services.

Example:

```
import { HttpClientModule } from '@angular/common/http';  
  
@NgModule({  
  imports: [HttpClientModule]  
})  
export class AppModule {}
```

25. How do you create a service in Angular?

To create a service in Angular, you can either manually create a service class or use the Angular CLI.

Using Angular CLI:

```
ng generate service user
```

This will create a `user.service.ts` file with the following structure:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root' // The service is available app-wide
})
export class UserService {
  constructor() { }
}
```

- **@Injectable() Decorator:** This marks the class as injectable, which means Angular can create instances of this service and inject it into components, other services, or directives.
- The `providedIn: 'root'` metadata specifies that the service is a singleton and will be available globally across the app.

Manually:

1. Create a service class with a method.
2. Add `@Injectable()` decorator.

Example:

```
import { Injectable } from '@angular/core';

@Injectable()
export class UserService {
  constructor() {}

  getUser() {
    return { name: 'John', age: 25 };
  }
}
```

Finally, you can inject the service into a component to use it.

26. What is the purpose of @Input and @Output decorators?

The @Input() and @Output() decorators are used for **communication between Angular components**.

@Input(): This decorator allows a **parent component** to pass data to a **child component**. Example:

```
@Component({
  selector: 'app-child',
  template: `<div>{{ parentData }}</div>`
})
export class ChildComponent {
  @Input() parentData: string;
}
```

The parent component can bind a value to the child component's @Input() property like this:

```
<app-child [parentData]="parentValue"></app-child>
```

@Output(): This decorator allows a **child component** to **emit an event** to its **parent component**. It's typically used for sending data or notifying the parent about an action. Example:

```
@Component({
  selector: 'app-child',
  template: `<button (click)="sendData()">Send Data</button>`
})
export class ChildComponent {
  @Output() dataEmitter = new EventEmitter<string>();

  sendData() {
    this.dataEmitter.emit('Hello, Parent!');
  }
}
```

The parent component listens for this event using the (dataEmitter) syntax:
html

Copy code

```
<app-child (dataEmitter)="receiveData($event)"></app-child>
```

27. What is Angular's change detection mechanism?

Angular's **change detection** mechanism is responsible for keeping the model and the view in sync. When a model changes, Angular updates the view to reflect those changes. It uses a strategy called **dirty checking** to determine if the model has changed.

There are two types of change detection strategies:

- **Default:** This strategy checks all components in the component tree for changes whenever an event occurs (e.g., user input, HTTP request, etc.).
- **OnPush:** This strategy optimizes performance by checking only components whose inputs have changed or whose events have triggered.

Change Detection Process:

1. Angular checks for changes in all bindings, such as input properties, template expressions, and observables.
2. If the values in the bindings change, Angular updates the DOM.

You can manually trigger change detection using `ChangeDetectorRef` or `ApplicationRef`.

28. What is the purpose of ngContent?

ngContent is used in Angular to implement **content projection**, which is a mechanism that allows you to insert dynamic content into a component's template. It is similar to **slots** in web components or custom elements.

With **ngContent**, you can insert arbitrary content from the parent component into the child component's view.

Example of **ngContent**:

Parent component:

```
<app-card>
  <h1>Dynamic Content</h1>
  <p>This content will be projected into the card component.</p>
</app-card>
```

Child component (app-card):

```
<div class="card">  
  <ng-content></ng-content>  
</div>
```

The content inside the <app-card> tag in the parent component will be projected into the ng-content tag of the child component.

29. What is an Angular Pipe?

An **Angular Pipe** is a way to transform data in the template, allowing developers to modify the data before displaying it to the user. Pipes can transform strings, numbers, dates, or arrays into desired formats.

Pipes are used within template expressions, and they are composed using the | symbol.

Common Angular Pipes:

- **DatePipe**: Transforms dates into a specified format.
- **UpperCasePipe**: Transforms text to uppercase.
- **LowerCasePipe**: Transforms text to lowercase.
- **CurrencyPipe**: Formats a number as a currency.
- **AsyncPipe**: Unwraps an observable or promise.

Example:

```
<p>{{ userName | uppercase }}</p>
```

You can also create custom pipes by implementing the PipeTransform interface.

30. What is the async pipe in Angular?

The **async pipe** is a special Angular pipe used to subscribe to **Observables** or **Promises** in the template. It automatically handles subscription and unsubscription, making it convenient for working with asynchronous data streams without the need for explicit subscription logic in the component.

Example:

If you have an observable `users$` in your component, instead of subscribing manually, you can use the async pipe in the template:

```
<ul>
  <li *ngFor="let user of users$ | async">{{ user.name }}</li>
</ul>
```

The async pipe automatically subscribes to `users$` and updates the view when new data arrives. It also handles the unsubscription when the component is destroyed.

31. How do you bind events in Angular?

In Angular, you can bind events using the **Event Binding** syntax, which allows you to listen to DOM events and trigger methods or expressions in your component.

Event binding syntax is as follows:

```
<button (click)="onClick()">Click Me</button>
```

Here:

- The click event is bound to the `onClick()` method in the component.
- When the button is clicked, Angular will call the `onClick()` method.

Passing Parameters to Event Handlers:

You can also pass parameters to event handler methods:

```
<button (click)="onClick($event)">Click Me</button>
```

In your component, you can handle the `$event` (which contains information about the event):

```
onClick(event: MouseEvent) {
  console.log('Button clicked!', event);
}
```

Event Binding with Method Expressions:

You can use expressions directly in the event binding if you don't need to define a separate method:

```
<button (click)="count = count + 1">Increment</button>
```

32. What is the difference between a component and a directive in Angular?

Both **components** and **directives** are fundamental building blocks of Angular applications, but they serve different purposes.

- **Component:**
 - A component is a **UI element** that controls a section of the screen (a view).
 - A component consists of an HTML template, a CSS style, and a TypeScript class that defines the logic.
 - Components are used to display content and manage views in Angular applications.
 - They are typically responsible for handling user interaction and rendering templates.

Example of a component:

```
@Component({  
  selector: 'app-user',  
  template: `<h1>{{ userName }}</h1>`  
})  
export class UserComponent {  
  userName: string = 'John Doe';  
}
```

- **Directive:**
 - A directive is a **behavioral** class that can modify the behavior of an element, component, or another directive.
 - Directives don't have their own view or template; they are applied to DOM elements or components to modify or extend their behavior.
 - There are three types of directives: **Structural**, **Attribute**, and **Component** (which is technically a special kind of directive).

Example of an attribute directive:

```
@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  constructor(private el: ElementRef) {
    el.nativeElement.style.backgroundColor = 'yellow';
  }
}
```

33. How do you perform validation in Angular forms?

Validation in Angular forms can be done in two ways: **Template-Driven Forms** and **Reactive Forms**. Both have mechanisms to apply validations, but they are used in different contexts.

In Template-Driven Forms:

You use built-in directives such as `ngModel` with validation attributes like `required`, `minlength`, etc.

Example:

```
<form #myForm="ngForm">
  <input name="username" [(ngModel)]="username" required
  minlength="5">
  <div *ngIf="myForm.submitted && myForm.controls.username.errors">
    <span *ngIf="myForm.controls.username.errors.required">Username
    is required</span>
    <span *ngIf="myForm.controls.username.errors.minlength">Username
    must be at least 5 characters</span>
  </div>
  <button [disabled]="!myForm.valid">Submit</button>
</form>
```

In this example:

- The input field is required and must have at least 5 characters.
- Validation errors are shown only after the form is submitted.

In Reactive Forms:

Reactive forms provide more flexibility by defining validation directly in the form model (in TypeScript).

Example:

```
import { FormBuilder, Validators } from '@angular/forms';

constructor(private fb: FormBuilder) {}

form = this.fb.group({
  username: ['', [Validators.required, Validators.minLength(5)]]
});
```

In the template:

```
<form [formGroup]="form" (ngSubmit)="onSubmit()">
  <input formControlName="username">
  <div *ngIf="form.get('username').invalid &&
form.get('username').touched">
    <span *ngIf="form.get('username').hasError('required')">Username
is required</span>
    <span *ngIf="form.get('username').hasError('minlength')">Username
must be at least 5 characters</span>
  </div>
  <button [disabled]="form.invalid">Submit</button>
</form>
```

In reactive forms, you get more control over the form state and validation, especially in complex forms.

34. Explain the difference between template-driven and reactive forms in Angular.

- **Template-Driven Forms:**
 - **Easy to set up:** These forms are based on Angular's directives and templates.

- **Minimal TypeScript code:** Most of the form logic is written in the template using ngModel.
- **Two-way data binding:** Forms are easier to implement with two-way data binding ([[(ngModel)]]).
- **Synchronous Validation:** Validation is declared within the template.

Example:

```
<form #form="ngForm">
  <input name="username" [(ngModel)]="username" required>
</form>
```

- **Reactive Forms:**
 - **More control:** Reactive forms are based on **form models** that are entirely defined in the component.
 - **Declarative:** You define the form structure and validation rules in TypeScript.
 - **Asynchronous Validation:** Reactive forms support both synchronous and asynchronous validators.
 - **Easier to scale:** Ideal for complex forms and larger applications with dynamic form controls.

Example:

```
import { FormGroup, FormBuilder, Validators } from '@angular/forms';

this.form = this.fb.group({
  username: ['', [Validators.required, Validators.minLength(5)]]
});
```

Summary:

- Template-driven forms are simpler to use, with most of the logic defined in the HTML template.
- Reactive forms provide more flexibility and control, especially for complex forms and dynamic validations.

35. What are Angular modules and why are they important?

Angular Modules are a way to organize an Angular application into cohesive blocks of functionality. An Angular application is made up of one or more modules, and each module can contain components, services, pipes, and directives that are related to a specific part of the application.

Key points about Angular modules:

- **Root Module:** The AppModule is the root module of any Angular application. It's where the application starts.
- **Feature Modules:** These modules contain features or functionality, such as a module for user management, authentication, or settings.
- **Shared Module:** A module that contains common components, directives, pipes, and services that can be shared across the application.
- **Lazy Loaded Modules:** Feature modules can be lazy-loaded to improve application startup time.

Example:

```
@NgModule({
  declarations: [AppComponent, UserComponent],
  imports: [CommonModule, HttpClientModule],
  providers: [UserService],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Modules help with:

- **Separation of concerns:** Organizing different features and concerns into different parts of the application.
- **Lazy loading:** Loading parts of the application only when needed, improving performance.
- **Reusability:** Reusing code (components, services) across different parts of the application.

36. What is lazy loading in Angular?

Lazy Loading is a technique in Angular where feature modules are loaded only when they are needed, rather than when the application starts. This helps improve the application's initial load time and overall performance.

With lazy loading, Angular will load only the necessary code for the current route and delay loading other code until the user navigates to a route that requires it.

To implement lazy loading in Angular:

1. Define a feature module.
2. Set up routing to lazy-load the module using the loadChildren property.

Example:

```
// app-routing.module.ts
const routes: Routes = [
  { path: 'user', loadChildren: () =>
import('./user/user.module').then(m => m.UserModule) }
];
```

This configuration ensures that the UserModule is only loaded when the user navigates to the /user route.

37. What is a subscription in Angular and how do you unsubscribe?

In Angular, a **subscription** is used to subscribe to an **Observable**, such as when making HTTP requests, listening to events, or handling async data.

When you subscribe to an observable, it begins to emit values. To avoid memory leaks or unnecessary background tasks, it's essential to unsubscribe when the component or service is destroyed.

Example of Subscription:

```
import { Observable } from 'rxjs';

const observable = new Observable(observer => {
  observer.next('Hello');
});

const subscription = observable.subscribe(value => {
```

```
    console.log(value); // Output: 'Hello'  
  });
```

Unsubscribing:

To unsubscribe, call the `unsubscribe()` method on the subscription:

```
subscription.unsubscribe();
```

In Angular, you typically unsubscribe in the `ngOnDestroy()` lifecycle hook to avoid memory leaks:

```
ngOnDestroy() {  
  this.subscription.unsubscribe();  
}
```

Alternatively, use the **async pipe** in templates to automatically manage subscriptions.

38. How does Angular handle cross-site scripting (XSS)?

Angular helps protect applications from **Cross-Site Scripting (XSS)** attacks through its **built-in security mechanisms**, particularly **sanitizing user input**.

1. **Automatic HTML sanitization:** Angular automatically sanitizes any untrusted content to prevent potentially dangerous code execution. For example, when binding HTML content dynamically with `{{ }}`, Angular escapes dangerous characters and prevents malicious scripts from running.
2. **DomSanitizer:** If you need to insert dynamic HTML (e.g., user input) into the DOM, Angular provides the DomSanitizer service to sanitize the content and mark it as safe.

```
import { DomSanitizer, SafeHtml } from '@angular/platform-browser';  
  
constructor(private sanitizer: DomSanitizer) {}  
  
getSafeHtml(unsafeHtml: string): SafeHtml {  
  return this.sanitizer.bypassSecurityTrustHtml(unsafeHtml);  
}
```

```
}
```

Angular's default **[sanitization strategy]** prevents **XSS attacks** by making sure that any untrusted HTML content is not directly rendered.

39. What is the purpose of the @angular/forms module?

The **@angular/forms** module provides functionality for handling user input through forms in Angular. It contains APIs to work with both **template-driven** and **reactive** forms. This module enables features such as:

- Form controls and validation
- Handling dynamic forms
- Managing form state
- Managing form submission and tracking validation errors

The module includes services, directives, and classes like FormControl, FormGroup, Validators, ReactiveFormsModule, and FormsModule.

To use it, import the relevant form module in your app:

```
import { ReactiveFormsModule } from '@angular/forms';
@NgModule({
  imports: [ReactiveFormsModule]
})
export class AppModule {}
```

40. What is the ng serve command in Angular?

The **ng serve** command is used to run an Angular application in development mode. It compiles the application and serves it on a local development server, allowing you to view the application in the browser.

Features of ng serve:

- **Live-reloading:** It automatically reloads the page in the browser when you make changes to the code.
- **Development mode:** Runs the application in development mode with optimizations turned off.
- **Default port:** By default, it runs on http://localhost:4200.

You can use `ng serve` with optional flags, such as:

- `--port`: to specify a custom port.
- `--open`: to automatically open the browser when the server starts.

Example:

```
ng serve --port 4201 --open
```

This will serve your Angular app on `http://localhost:4201` and open it in the default browser.

Intermediate (Q&A)

1. Explain the difference between `ngOnInit()` and `constructor()` in Angular.

Both `ngOnInit()` and `constructor()` are important parts of an Angular component's lifecycle, but they serve different purposes.

- **`constructor()`:**
 - The `constructor()` is a special method in a class used for **initializing class members**.
 - It's called when the class is instantiated (i.e., when the component is created).
 - Typically used to inject dependencies via Angular's **Dependency Injection (DI)** system, such as services.
 - **Not ideal** for initializing data or triggering any behavior tied to the Angular lifecycle.

Example:

```
constructor(private myService: MyService) {  
  // Used for DI and setting up properties  
}
```

- **`ngOnInit()`:**
 - The `ngOnInit()` is a lifecycle hook method that **Angular calls once the component's data-bound properties have been initialized**.

- It is often used for **initializing data** or performing any other setup that requires **bindings** to be set.
- `ngOnInit()` is called after the constructor and is more suitable for **component initialization** logic.

Example:

```
ngOnInit(): void {  
    // This is where you can fetch data or perform other initialization  
    tasks  
    this.fetchData();  
}
```

In summary:

- **Constructor** is for dependency injection and setting up class properties.
- **ngOnInit()** is for **initializing data** or executing code that depends on bindings and inputs being set.

2. What are the various lifecycle hooks in Angular, and how are they are used?

Angular provides several lifecycle hooks that allow developers to run custom logic during specific stages of a component's lifecycle. Here's a list of key lifecycle hooks:

1. `ngOnChanges()`:

- Called when any data-bound input properties change.
- Useful for responding to changes in inputs passed to a component.

Example:

```
ngOnChanges(changes: SimpleChanges) {  
    console.log(changes);  
}
```

1. `ngOnInit()`:

- Called once, after the component's data-bound properties have been initialized.
- Ideal for initializing data or triggering actions that require inputs to be set.

2. `ngDoCheck()`:

- Called during every change detection cycle.
- Used for implementing custom change detection if needed.

3. **ngAfterContentInit():**

- Called after Angular has projected content into the component (i.e., after content passed via `<ng-content>` is initialized).
- Useful for interacting with projected content.

4. **ngAfterContentChecked():**

- Called after the content has been checked during change detection.
- Allows you to check and respond to changes in projected content.

5. **ngAfterViewInit():**

- Called once after the component's view (and child views) has been initialized.
- Typically used to interact with view elements, such as after initializing child components or querying the DOM.

6. **ngAfterViewChecked():**

- Called after the view has been checked by Angular's change detection.
- Can be used to respond to changes in the view.

7. **ngOnDestroy():**

- Called just before the component is destroyed.
- Useful for cleanup, such as unsubscribing from observables or detaching event handlers.

Example:

```
ngOnDestroy() {  
  console.log("Component is being destroyed");  
}
```

Each of these lifecycle hooks provides opportunities for executing code at various points during the lifecycle of a component or directive.

3. What is an Angular decorator? Name a few commonly used decorators.

An **Angular decorator** is a special kind of function that modifies the behavior of a class or its members. Decorators are used to **add metadata** to classes, methods, properties, or parameters in Angular.

Some commonly used Angular decorators:

1. **@Component():**

- Marks a class as an Angular component and provides metadata like the component's template, selector, and styles.

```
@Component({  
  selector: 'app-root',  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
export class AppComponent { }
```

1. **@NgModule():**

- Defines an Angular module, providing metadata that describes how the module's components, services, and other parts fit together.

```
@NgModule({  
  declarations: [AppComponent],  
  imports: [BrowserModule],  
  providers: [],  
  bootstrap: [AppComponent]  
})  
export class AppModule { }
```

1. **@Injectable():**

- Marks a class as injectable, which means Angular can create instances of the class through Dependency Injection.

```
@Injectable({  
  providedIn: 'root'  
})  
export class MyService { }
```

1. **@Input():**

- Binds a property to an input of a component, allowing parent components to pass data into a child component.

```
@Input() userName: string;
```

1. @Output():

- Marks a property as an output, allowing a child component to emit events to a parent component.

```
@Output() update = new EventEmitter<string>();
```

1. @ViewChild():

- Queries a child component or DOM element in the component's view.

```
@ViewChild(MyComponent) child: MyComponent;
```

4. What is a factory provider in Angular?

A **Factory Provider** is a way of defining custom logic to create an instance of a service in Angular. Instead of using a simple class to provide a service, a factory function is used to create the service instance.

A factory provider is useful when:

- You need to create a service instance dynamically.
- You need to use some logic or conditions to instantiate the service.
- You need to provide a mock or alternative service in testing.

Example:

```
export function loggerFactory() {
  return new LoggerService('my-app');
}

@NgModule({
  providers: [
    { provide: LoggerService, useFactory: loggerFactory }
  ]
})
export class AppModule {}
```

In this example, the `loggerFactory` function is used to create the `LoggerService` instance.

5. What is Angular's Change Detection Strategy?

Change detection in Angular is the process through which Angular checks the application's model and updates the view whenever the model changes. Angular has two change detection strategies:

1. Default Change Detection Strategy:

- Angular checks all components in the component tree whenever a change occurs (e.g., when a user interacts, when a network response is received).
- Every event triggers a full re-evaluation of the component tree.

2. OnPush Change Detection Strategy:

- Angular will only check components for changes if:
 - The component's input properties change.
 - An event (such as a user click) happens within the component.
 - The component is manually triggered to check for changes.
- 3. This strategy can improve performance, particularly in large applications, because Angular doesn't check all components on every change detection cycle.

Example:

```
@Component({
  selector: 'app-user',
  changeDetection: ChangeDetectionStrategy.OnPush,
  templateUrl: './user.component.html'
})
export class UserComponent { }
```

6. What is ngZone in Angular, and why is it important?

ngZone is an Angular service that allows Angular to **track asynchronous operations** (such as HTTP requests, timers, and promises) and update the view when the operation completes. It ensures that Angular is aware of any changes made outside Angular's change detection context, like `setTimeout` or `setInterval`.

Why it's important:

- Angular uses zones to automatically detect and respond to changes from asynchronous operations.
- **Without ngZone**, Angular would not know when to update the DOM after asynchronous operations, leading to inconsistencies in the UI.

You can interact with ngZone directly to control when change detection runs or bypass it entirely.

```
import { NgZone } from '@angular/core';

constructor(private ngZone: NgZone) {}

someAsyncOperation() {
  this.ngZone.run(() => {
    // This triggers change detection when the async operation
    // completes
  });
}
```

7. Explain the use of @ViewChild and @ContentChild.

- **@ViewChild():**
 - @ViewChild allows you to query and access a **child component or DOM element** that is part of the component's view (directly inside the component's template).
 - Useful for interacting with child components, getting references to DOM elements, or controlling the child component's properties.

Example:

```
@ViewChild(MyComponent) child: MyComponent;
```

- **@ContentChild():**
 - @ContentChild allows you to access a **child element** that is projected into the component using ng-content.
 - It queries the **content projected into the component** by the parent (via ng-content).

Example:

```
@ContentChild(MyDirective) contentChild: MyDirective;
```

8. How can you optimize the performance of an Angular application?

There are several techniques for optimizing Angular application performance:

1. Use OnPush Change Detection:

- By using the OnPush change detection strategy, Angular will only check components when an input property changes or an event occurs inside the component.

2. Lazy Loading:

- Load modules only when required, rather than loading the entire application at once.

3. Avoid Complex Expressions in Templates:

- Avoid performing complex logic inside templates, as this can trigger frequent change detection cycles.

4. Track by in ngFor:

- Use the trackBy function to optimize rendering of list items.

5. Tree Shaking:

- Use **AOT (Ahead of Time) Compilation** to reduce the size of the bundle by removing unused code.

6. Debounce User Input:

- Use debouncing for user inputs like search bars to reduce the number of API calls.

7. Service Workers:

- Implement service workers to cache assets and API responses for offline usage.

8. Minification and Compression:

- Minify and compress JavaScript and CSS files for faster load times.

9. What is the trackBy function in ngFor and why is it important?

The **trackBy** function in Angular's ngFor directive helps Angular track items in lists more efficiently by identifying each item uniquely, preventing unnecessary re-rendering when the list changes.

By default, Angular re-renders every item in the list when the data changes. With trackBy, you can specify a unique identifier for each item, improving performance for large lists.

Example:

```
<ul>
  <li *ngFor="let item of items; trackBy: trackById">{{ item.name }}
</li>
</ul>
```

```
trackById(index: number, item: any): number {
  return item.id; // Unique identifier for each item
}
```

This way, Angular only updates items whose data has changed, instead of re-rendering the entire list.

10. What are Angular interceptors, and how are they used?

Angular interceptors are services that implement the `HttpInterceptor` interface and allow you to **intercept and modify HTTP requests** and responses globally, before they are sent or after they are received.

Common use cases for interceptors:

- Adding authentication tokens to headers.
- Logging HTTP requests and responses.
- Handling errors globally.
- Modifying response data.

Example:

```
@Injectable()
```

```
export class AuthInterceptor implements HttpInterceptor {  
  intercept(req: HttpRequest<any>, next: HttpHandler):  
    Observable<HttpEvent<any>> {  
    const clonedReq = req.clone({  
      setHeaders: { Authorization: 'Bearer ' +  
this.authService.getToken() }  
    });  
    return next.handle(clonedReq);  
  }  
}
```

To register the interceptor, add it to the providers array in the module:

```
@NgModule({  
  providers: [  
    { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi:  
true }  
  ]  
})  
export class AppModule {}
```

Interceptors provide a powerful way to centralize HTTP logic, such as authentication or error handling.

11. Explain the concept of Observables and Promises in Angular.

Both **Observables** and **Promises** are used for handling asynchronous operations, but they have significant differences in their behavior, use cases, and capabilities.

- **Promises:**
 - A **Promise** represents a single asynchronous operation that completes in the future with either a value or an error.
 - Once a Promise is resolved (or rejected), it can never change its state, i.e., it is **immutable**.
 - **Promises** are eager, meaning they start executing as soon as they are created.
 - They are useful when you need to handle a **single value** asynchronously (like a single HTTP request).

Example:

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => resolve("Success"), 1000);  
});  
  
promise.then(result => console.log(result)); // Logs "Success" after  
1 second
```

- **Observables:**

- An **Observable** is a more powerful abstraction, representing a **stream of values** over time, which can emit multiple values or errors. It can be cancelled or unsubscribed from.
- Unlike Promises, Observables are **lazy**, meaning they only execute when subscribed to.
- Observables are a key part of **Reactive Programming** and are used extensively with **RxJS (Reactive Extensions for JavaScript)**.
- Observables can emit multiple values (for example, an HTTP request that receives data over time, or user inputs).

Example:

```
import { Observable } from 'rxjs';  
  
const observable = new Observable(subscriber => {  
  setTimeout(() => subscriber.next("First Value"), 1000);  
  setTimeout(() => subscriber.next("Second Value"), 2000);  
});  
  
observable.subscribe(value => console.log(value));  
// Logs "First Value" after 1 second, then "Second Value" after 2  
seconds
```

Key Differences:

- **Promises** handle a single event (resolved or rejected) while **Observables** can handle multiple events over time.
- **Promises** are **eager** (executed immediately when defined) while **Observables** are **lazy** (executed only when subscribed to).
- **Promises** have only a **single value**, while **Observables** can emit multiple values over time.

In Angular, Observables are often used for asynchronous operations like HTTP requests, handling streams of user inputs, or handling events.

12. What is the purpose of the RxJS library in Angular?

RxJS (Reactive Extensions for JavaScript) is a library for **reactive programming** using **Observables**, making it easier to compose asynchronous or callback-based code using operators such as `map`, `filter`, `merge`, etc.

In Angular, **RxJS** plays a central role because:

- It is used extensively in Angular's HTTP client for **handling asynchronous data** (such as HTTP requests and responses).
- Angular uses **Observables** for event handling, form validation, and many other async operations.
- RxJS enables powerful operations like **debouncing**, **retrying failed requests**, **combining multiple streams of data**, and more.

Common operators include:

- `map()`, `filter()`, `mergeMap()`, `switchMap()`, `take()`, and `catchError()`.

Example of using RxJS with Angular HTTP Client:

```
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  constructor(private http: HttpClient) {}

  fetchData(): Observable<any> {
    return this.http.get('https://api.example.com/data').pipe(
      map(response => response['data']),
      catchError(error => {
        console.error('Error fetching data', error);
        throw error;
      })
    );
  }
}
```

```

    );
  }
}

```

RxJS helps manage async data streams efficiently and is a powerful tool to make Angular applications more reactive and maintainable.

13. How does Angular handle forms validation with custom validators?

Angular provides built-in form validators, but custom validators can be defined when more complex validation is required. Custom validators are functions that return an **error object** or null if the validation passes. Custom validators can be used in both **template-driven** forms and **reactive** forms.

Custom Validator for Template-Driven Forms:

```

import { AbstractControl, ValidationErrors } from '@angular/forms';

// Custom Validator
export function forbiddenNameValidator(control: AbstractControl):
ValidationErrors | null {
  const forbiddenNames = ['admin', 'root'];
  return forbiddenNames.includes(control.value) ? { forbiddenName: {
value: control.value } } : null;
}

```

To apply the custom validator in a template-driven form:

```

<input [(ngModel)]="username" name="username" [ngModelOptions]="
{standalone: true}" [ngModel]="username"
[forbiddenNameValidator]="true">

```

Custom Validator for Reactive Forms:

```

import { FormGroup, FormBuilder, Validators } from '@angular/forms';

// Define a form with a custom validator
this.form = this.fb.group({
  username: ['', [Validators.required, forbiddenNameValidator]]

```

```
});
```

Custom validators can return an error object (such as { forbiddenName: { value: 'admin' } }) if the validation fails, or null if the input is valid.

14. What is the difference between `ngOnInit()` and `ngAfterViewInit()`?

- **`ngOnInit()`:**
 - Called once after the component's inputs have been initialized (i.e., after Angular sets up data bindings).
 - Ideal for **initializing data**, performing API calls, or setting up properties that depend on the component's inputs.

Example:

```
ngOnInit() {  
  console.log('Component initialized!');  
  // Initialize data or perform actions on inputs  
}
```

- **`ngAfterViewInit()`:**
 - Called once after the component's view (and child views) has been initialized. This includes child components, DOM elements, and directives.
 - Ideal for any **DOM manipulations** or interactions with **view-related elements**.
 - `ngAfterViewInit()` is called after Angular has initialized all child components in the view, unlike `ngOnInit()` which is called before view initialization.

Example:

```
ngAfterViewInit() {  
  console.log('View and child views initialized!');  
  // Access or manipulate DOM elements in the view  
}
```

Key Differences:

- `ngOnInit()` is for **initializing data**, while `ngAfterViewInit()` is for interacting with the **DOM** and **view elements** (child components and templates).

15. What are lazy-loaded modules, and how do they work in Angular?

Lazy loading is a design pattern used to load **modules or components only when they are needed** (on demand) rather than loading them upfront when the application starts. This can significantly improve the application's initial load time by splitting the application into smaller chunks.

To implement lazy loading in Angular, use the **Angular Router** to define routes that load modules only when the user navigates to a certain route.

Example:

1. Create a feature module (e.g., `FeatureModule`).

In the `app-routing.module.ts`, use `loadChildren` to load the module lazily:

```
const routes: Routes = [  
  {  
    path: 'feature',  
    loadChildren: () => import('./feature/feature.module').then(m =>  
      m.FeatureModule)  
  }  
];
```

In this example, the **FeatureModule** will be loaded only when the user navigates to the `/feature` route, reducing the initial bundle size.

16. What is NgModule in Angular, and how does it help with modularity?

An **NgModule** is a fundamental building block in Angular that provides a mechanism to **group related components, directives, pipes, and services** together. Modules help organize the codebase into logical, reusable blocks, and they define the boundaries of functionality within the app.

Key features of NgModule:

- **Declarations:** Specifies the components, directives, and pipes that belong to the module.
- **Imports:** Defines other modules that the current module depends on.

- **Providers:** Defines services or other providers that are available to the components in this module.
- **Bootstrap:** Specifies the root component that Angular should bootstrap when the application starts.

Example of an NgModule:

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

By using **NgModules**, Angular provides a way to logically organize and modularize an application, making it easier to scale, maintain, and reuse code.

17. How do you pass data between Angular components?

Data can be passed between Angular components in several ways:

1. **Parent to Child (via @Input):**
 - Use the @Input() decorator in the child component to bind properties passed from the parent.

Example:

```
// Parent Component
@Component({ selector: 'app-parent' })
export class ParentComponent {
  message = 'Hello from Parent';
}

// Child Component
@Component({ selector: 'app-child' })
export class ChildComponent {
  @Input() message: string;
}
```

```
<!-- Parent Template -->
<app-child [message]="message"></app-child>
```

1. Child to Parent (via @Output and EventEmitter):

- Use the @Output() decorator in the child component with an EventEmitter to send events or data back to the parent.

Example:

```
// Child Component
@Component({ selector: 'app-child' })
export class ChildComponent {
  @Output() notifyParent = new EventEmitter<string>();

  sendMessage() {
    this.notifyParent.emit('Hello from Child');
  }
}

// Parent Component
@Component({ selector: 'app-parent' })
export class ParentComponent {
  onMessageReceived(message: string) {
    console.log(message);
  }
}
```

```
<!-- Parent Template -->
<app-child (notifyParent)="onMessageReceived($event)"></app-child>
```

1. Using a Service (for sibling components or between any components):

- A shared service can be used to store and transmit data between any components via Observables.

18. What are Angular Guards, and what are their different types?

Angular **Guards** are used to control access to routes, allowing you to protect routes based on certain conditions. There are several types of guards in Angular:

1. **CanActivate:**

- Determines whether a route can be activated (i.e., whether a user can navigate to it).
- Used for authentication or authorization checks.

Example:

```
canActivate(): boolean {  
  return this.authService.isAuthenticated();  
}
```

1. **CanDeactivate:**

- Prevents a user from navigating away from a route if there are unsaved changes.

Example:

```
canDeactivate(): boolean {  
  return confirm('Are you sure you want to leave without saving?');  
}
```

1. **CanLoad:**

- Prevents a module from being loaded lazily until certain conditions are met (e.g., authorization).

Example:

```
canLoad(): boolean {  
  return this.authService.hasPermission();  
}
```

1. **Resolve:**

- Retrieves data before a route is activated. It allows resolving data asynchronously before entering a route.

Example:

```
resolve(route: ActivatedRouteSnapshot): Observable<any> {
```

```

    return this.dataService.getData();
  }
}

```

19. How can you handle errors in Angular HTTP requests?

You can handle errors in Angular HTTP requests by using the **catchError** operator from RxJS. This allows you to catch HTTP errors and perform custom error handling.

Example:

```

import { HttpClient } from '@angular/common/http';
import { catchError } from 'rxjs/operators';
import { throwError } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  constructor(private http: HttpClient) {}

  fetchData() {
    return this.http.get('https://api.example.com/data').pipe(
      catchError(error => {
        console.error('Error occurred:', error);
        return throwError('Something went wrong!'); // Or a custom
error handling logic
      })
    );
  }
}

```

In this example, **catchError** intercepts the error and provides a fallback or custom error message.

20. What is the role of the HttpClientModule and how do you use it?

The **HttpClientModule** is an Angular module that provides a simplified API for making HTTP requests. It is part of Angular's `@angular/common/http` package and allows you to interact with remote APIs using **Observables**.

To use HttpClientModule:

1. Import `HttpClientModule` into your root module (`AppModule`).
2. Inject `HttpClient` in your service to make HTTP requests.

Example:

```
import { HttpClientModule } from '@angular/common/http';
import { HttpClient } from '@angular/common/http';

@NgModule({
  imports: [HttpClientModule],
})
export class AppModule { }

@Injectable({
  providedIn: 'root'
})
export class DataService {
  constructor(private http: HttpClient) {}

  getData() {
    return this.http.get('https://api.example.com/data');
  }
}
```

The `HttpClient` simplifies HTTP operations, making it easier to handle responses, errors, and HTTP methods (GET, POST, PUT, DELETE, etc.).

21. Explain the difference between `HttpClient` and `Http` in Angular.

In Angular, `HttpClient` and `Http` are both used to make HTTP requests, but `HttpClient` is the newer, preferred API that offers several improvements over the older `Http` API.

- **HttpClient** (Introduced in Angular 4.3):
 - A more modern API that is **simplified**, and returns **Observable** objects, making it more compatible with RxJS and easier to work with for reactive programming.
 - **HttpClient** supports features such as **request/response interception**, **typed responses**, and **progress events**.
 - Automatically parses responses as JSON (if the response body is JSON), eliminating the need for manual JSON parsing.

- **HttpClient** is provided by the `@angular/common/http` module and must be imported separately.
- **Http** (Deprecated since Angular 4.3):
 - The older service for making HTTP requests, which used **RxJS Observables** but had a less intuitive API.
 - **Http** required manual handling of JSON parsing for most requests.
 - **Http** is still available in older versions of Angular but should not be used for new development, as it is now deprecated.

Example:

```
// HttpClient (preferred):
import { HttpClient } from '@angular/common/http';

this.httpClient.get('/api/data').subscribe(response => {
  console.log(response);
});

// Http (deprecated):
import { Http } from '@angular/http';

this.http.get('/api/data').subscribe(response => {
  console.log(response.json());
});
```

22. What is the role of the Injector in Angular?

The **Injector** in Angular is a core concept of the **dependency injection (DI)** system. It is responsible for creating and providing instances of services (or other dependencies) to Angular components, directives, pipes, and services. The Injector manages the lifecycle of services and determines how and when they are instantiated and injected into the class constructors.

- **Providers** in Angular are registered with a specific injector, and when the Angular component or service needs a dependency, the injector is responsible for providing it.

- **Hierarchy of Injectors:** Angular has a **hierarchical injector system**, where child components inherit injectors from their parent components. This allows Angular to manage the scope and lifetime of services at various levels of the application.

Example:

```
@Injectable({
  providedIn: 'root' // Service provided at the root level of the
  injector
})
export class MyService {
  constructor() { }
}
```

Here, Angular will use the **root injector** to provide the instance of MyService throughout the application.

23. How does Angular handle routing?

Angular's routing mechanism allows you to define routes that map URLs to components. The **Router** provides navigation and enables dynamic updates of views without reloading the entire page.

Key Concepts in Angular Routing:

1. **RouterModule:** To enable routing, you must import **RouterModule** in your application module and define routes using the **Routes** array.
2. **Routes:** Define paths (URLs) and map them to Angular components. Each route is associated with a component that will be displayed when that route is activated.
3. **RouterLink:** A directive used in templates to link to different routes.
4. **Router-outlet:** A placeholder directive where the routed component will be displayed.

Example:

1. Define routes in app-routing.module.ts:

```
const routes: Routes = [  
  { path: 'home', component: HomeComponent },  
  { path: 'about', component: AboutComponent }  
];
```

2. Import RouterModule in AppModule:

```
@NgModule({  
  imports: [RouterModule.forRoot(routes)],  
  exports: [RouterModule]  
})  
export class AppRoutingModule {}
```

3. Use routerLink in templates:

```
<a routerLink="/home">Home</a>  
<a routerLink="/about">About</a>  
<router-outlet></router-outlet> <!-- Routed component will be  
inserted here -->
```

24. What is an Angular resolver, and when would you use it?

An **Angular Resolver** is a service that is used to **pre-fetch data** before a route is activated. Resolvers are commonly used to fetch data from an API or perform other asynchronous tasks before the component for a specific route is loaded. This ensures that the component is rendered with the necessary data from the start.

Key Use Cases for Resolvers:

- To resolve data required for a route before the view is displayed.
- To avoid showing incomplete or empty views while waiting for data to be fetched.
- To handle complex data fetching logic before rendering a component.

Example:

1. Create a resolver:

```
@Injectable({
  providedIn: 'root'
})
export class DataResolver implements Resolve<any> {
  constructor(private dataService: DataService) {}

  resolve(route: ActivatedRouteSnapshot, state: RouterStateSnapshot):
  Observable<any> {
    return this.dataService.fetchData();
  }
}
```

2. Configure the resolver in routes:

```
const routes: Routes = [
  {
    path: 'data',
    component: DataComponent,
    resolve: {
      data: DataResolver
    }
  }
];
```

Now, DataComponent will only be activated once the data has been fetched via DataResolver.

25. What is ngModel used for in Angular forms?

ngModel is a directive used to create two-way data binding in Angular forms. It binds an input field (or other form elements) to a component's property. It keeps the view (HTML) and the model (component data) in sync. This means that when the value of the input field changes, the component property is updated, and vice versa.

ngModel is typically used in template-driven forms for two-way binding.

Example:

```
<input [(ngModel)]="username" placeholder="Enter username">
```

In this case, when the user types into the input field, the username property in the component will automatically be updated with the value of the input field. Likewise, if the username property changes, the input field will update with the new value.

26. How do you create a custom pipe in Angular?

A **custom pipe** in Angular allows you to transform data in your templates. You can create your own pipe when you need to perform custom transformations that are not provided by the built-in pipes.

Steps to create a custom pipe:

1. Use the `@Pipe` decorator to define the pipe.
2. Implement the transform method to define how the input value should be transformed.
3. Use the custom pipe in a template.

Example:

1. Create the pipe:

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'reverse'
})
export class ReversePipe implements PipeTransform {
  transform(value: string): string {
    return value.split('').reverse().join('');
  }
}
```

2. Register the pipe in a module:

```
@NgModule({
  declarations: [ReversePipe],
})
export class AppModule { }
```

3. Use the pipe in a template:

```
<p>{{ 'Angular' | reverse }}</p>
```

This will display the text "ralugnA".

27. What are Angular modules, and how do they help in application structure?

Angular modules (NgModules) are the building blocks of an Angular application. They provide a way to organize the application's functionality into cohesive blocks of code. Each module can define components, directives, pipes, and services, and can import other modules to provide the necessary functionality.

The Key Benefits of Angular Modules:

1. **Modularity:** Allows developers to organize code in a logical manner.
2. **Reusability:** Components, directives, and services can be packaged into modules and reused across the application.
3. **Lazy Loading:** Modules can be loaded on demand, improving the performance of large applications.

Example:

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Here, AppModule is the root module of the application, which imports BrowserModule and declares the AppComponent.

28. What is the difference between `ngOnChanges()` and `ngDoCheck()` lifecycle hooks?

Both `ngOnChanges()` and `ngDoCheck()` are lifecycle hooks in Angular, but they serve different purposes:

- **`ngOnChanges()`:**
 - Called whenever any **input-bound property** of a component changes. It is triggered when the **inputs** to the component are changed by the parent component.
 - Ideal for reacting to changes in the component's input properties.

Example:

```
ngOnChanges(changes: SimpleChanges) {  
  console.log(changes);  
}
```

- **`ngDoCheck()`:**
 - Called during every **change detection cycle**. It is used to detect and act upon changes that Angular cannot automatically detect.
 - It gives you full control over change detection and is useful when you need to track changes that Angular's default change detection strategy might not catch.

Example:

```
ngDoCheck() {  
  console.log('Change detection is running');  
}
```

Key Difference: `ngOnChanges()` is used for input property changes, while `ngDoCheck()` is a more general hook that is invoked every time Angular runs change detection.

29. Explain how the Angular module system works with imports, exports, and providers.

Angular's **module system** allows you to organize code into different functional blocks. The **imports**, **exports**, and **providers** are key aspects of how modules work:

Imports: Modules can import other modules to access their functionality. This allows you to use the components, directives, and services from other modules. Example:

```
imports: [CommonModule]
```

Exports: Modules can export components, directives, and pipes so that other modules can use them. Example:

```
exports: [MyComponent]
```

Providers: Modules can define **services** or other providers that can be injected into components, directives, or other services in the application. This helps in managing the **dependency injection system**. Example:

```
providers: [MyService]
```

30. How do you implement global error handling in Angular applications?

Global error handling in Angular is typically done using an **ErrorHandler** service. You can extend Angular's built-in **ErrorHandler** class to create a custom error handler for logging errors or displaying error messages globally.

Example:

1. Create a custom ErrorHandler:

```
import { Injectable, ErrorHandler } from '@angular/core';

@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  handleError(error: any): void {
    console.error('An error occurred:', error);
    // You can log the error to an external server or display a user-
    friendly message
  }
}
```

2. Provide the custom error handler in AppModule:

```
@NgModule({
  providers: [
    { provide: ErrorHandler, useClass: GlobalErrorHandler }
  ]
})
export class AppModule { }
```

With this approach, all uncaught errors in the application will be caught by the `GlobalErrorHandler` and handled appropriately.

31. How can you optimize large Angular applications?

Optimizing large Angular applications involves various techniques to improve performance, reduce memory consumption, and enhance user experience. Here are some key strategies:

1. Lazy Loading:

- Load modules only when needed. Lazy loading helps to split the application into smaller bundles, reducing the initial loading time and improving performance.

Use Angular's built-in **Router** to load modules lazily:

```
const routes: Routes = [
  {
    path: 'feature',
    loadChildren: () => import('./feature/feature.module').then(m =>
m.FeatureModule)
  }
];
```

1. Change Detection Strategy:

- By default, Angular uses the **ChangeDetectionStrategy.Default** where it checks the entire component tree for changes. You can optimize this by using **ChangeDetectionStrategy.OnPush**, which limits change detection to components with inputs that change or events that trigger updates.

2. Ahead-of-Time (AOT) Compilation:

- Enable AOT to compile your application during the build phase rather than runtime. This reduces the size of the bundle and improves performance.

3. **TrackBy in ngFor:**

- Use the **trackBy** function with ngFor to optimize rendering by helping Angular track which items in a list have changed, added, or removed.

Example:

```
<div *ngFor="let item of items; trackBy: trackById">
  {{ item.name }}
</div>
```

```
trackById(index: number, item: any): number {
  return item.id;
}
```

1. **Avoiding Unnecessary Change Detection:**

- Use **OnPush** change detection and immutability to minimize unnecessary re-renders.
- Avoid heavy operations in templates, such as complex expressions or calculations.

2. **Code Splitting:**

- Divide your application into multiple bundles (modules) and load them as needed. Use **Webpack** or Angular CLI's built-in features for this.

3. **Service Workers:**

- Use service workers for caching assets and API data to improve performance, especially for mobile devices with intermittent connectivity.

4. **Efficient Event Handling:**

- Debounce user input events (such as keystrokes) and avoid too many event listeners for better performance.

32. What is the difference between ngFor and ngForOf?

In Angular, **ngFor** and **ngForOf** are essentially the same, but there's a subtle difference in how they are used:

ngFor is a shorthand syntax for **ngForOf** and is the commonly used version. It is typically used as a structural directive to iterate over a list and render elements in the DOM. Example:

```
<div *ngFor="let item of items">
  {{ item }}
</div>
```

ngForOf is used when you want to explicitly reference the directive, which is less common. You might see it in more complex use cases or when you're manually adding directives to templates. Example:

```
<div *ngForOf="let item of items as itemsArray">
  {{ item }}
</div>
```

Although **ngForOf** exists, Angular's templating system encourages the use of **ngFor** in most cases because it is simpler and more concise.

33. How do you optimize Angular change detection using **ChangeDetectionStrategy**?

Angular uses **change detection** to keep the DOM in sync with the component data. By default, Angular uses **ChangeDetectionStrategy.Default**, which checks every component in the component tree during every change detection cycle.

To optimize change detection, you can use **ChangeDetectionStrategy.OnPush**, which tells Angular to check a component for changes only when:

1. The component's input properties change.
2. An event is triggered within the component.

This significantly reduces the number of components Angular needs to check, improving performance.

Example:

```
@Component({
  selector: 'app-optimized-component',
  changeDetection: ChangeDetectionStrategy.OnPush,
  templateUrl: './optimized.component.html',
})
export class OptimizedComponent {
  @Input() data: any;
}
```

When to use OnPush:

- Components that receive immutable data (e.g., objects or arrays that are not modified).
- Components where state is only modified via events or outputs, not directly by Angular change detection.

34. What are some performance best practices when working with Angular?

Here are some performance best practices for Angular:

1. **Use trackBy with ngFor:**
 - As mentioned earlier, trackBy helps Angular to identify which elements have changed, reducing the DOM manipulations.
2. **Lazy Loading:**
 - Implement lazy loading to load only the necessary parts of the application at first and defer loading of other parts until needed.
3. **Avoid Complex Templates:**
 - Avoid complex logic in templates and use **pure pipes** to handle data transformations in a more optimized manner.
4. **Use OnPush Change Detection:**
 - As discussed, use **ChangeDetectionStrategy.OnPush** to minimize the number of change detection cycles.
5. **Keep the Application Bundle Small:**
 - Minimize the size of your bundles by optimizing assets and using tree-shaking to remove unused code.

6. Defer Long-Running Operations:

- Use `setTimeout`, `setInterval`, or web workers to offload long-running tasks and avoid blocking the main thread.

7. Optimize Event Handling:

- Debounce user input, especially for inputs that trigger multiple events (like `keyup` and `keydown`).

8. Use Service Workers for Caching:

- Cache static assets and API responses to reduce network requests and improve load times for repeat visits.

9. Efficient Module Design:

- Design your application in a modular way so that Angular can lazy-load portions of it when necessary, reducing the initial load time.

35. What is an async pipe, and how is it used in Angular templates?

The **async pipe** in Angular automatically subscribes to an observable or a promise and unwraps the value for you. It manages subscriptions and automatically unsubscribes when the component is destroyed, helping you avoid memory leaks.

Example:

Suppose you have an observable that fetches data:

```
export class AppComponent {
  data$: Observable<Data>;

  constructor(private dataService: DataService) {
    this.data$ = this.dataService.getData();
  }
}
```

In your template, use the async pipe to bind the data:

```
<div *ngIf="data$ | async as data">
  <p>{{ data.name }}</p>
```

</div>

Here, the async pipe subscribes to `data$`, and when the data is available, it renders the content.

36. What is the difference between `HttpClient` and `HttpModule` in Angular?

- **HttpClient** is a modern and more feature-rich API for making HTTP requests, introduced in Angular 4.3. It is part of the `@angular/common/http` package and supports features such as **typed responses**, **interceptors**, **progress events**, and better error handling. It uses **Observables** for async handling.
- **HttpModule** is the older module used in Angular versions prior to 4.3. It provided basic HTTP functionality using **RxJS Observables** but lacks the features and improvements found in **HttpClient**.

HttpClient is the recommended choice for new Angular applications. To use it, you must import **HttpClientModule**.

Example:

```
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [HttpClientModule]
})
export class AppModule { }
```

37. What is the role of the `ngFor` directive in Angular?

The **ngFor** directive is used for iterating over a list or array of items and displaying them dynamically in the DOM. It creates an instance of the template for each item in the list, which is useful for displaying lists or collections of data.

Example:

```
<ul>
  <li *ngFor="let item of items">
    {{ item }}
  </li>
```


ngFor allows you to specify the item to iterate over and automatically creates the necessary DOM elements for each item in the list.

38. How can you dynamically load a component in Angular?

In Angular, you can dynamically load a component using **ViewContainerRef** and **ComponentFactoryResolver** or, in newer versions of Angular, the **ngComponentOutlet** directive.

Example:

Using ViewContainerRef and ComponentFactoryResolver (Angular 9 and earlier):

```
import { Component, ViewChild, ViewContainerRef,
ComponentFactoryResolver } from '@angular/core';
import { DynamicComponent } from './dynamic/dynamic.component';

@Component({
  selector: 'app-dynamic-loader',
  template: `<ng-template #container></ng-template>`
})
export class DynamicLoaderComponent {
  @ViewChild('container', { read: ViewContainerRef }) container:
  ViewContainerRef;

  constructor(private resolver: ComponentFactoryResolver) {}

  loadDynamicComponent() {
    const factory =
this.resolver.resolveComponentFactory(DynamicComponent);
    this.container.clear();
    this.container.createComponent(factory);
  }
}
```

39. Explain the concept of ngOnDestroy() and when to use it.

ngOnDestroy() is a lifecycle hook that is called when a component or directive is destroyed. It is commonly used for cleanup operations, such as:

- Unsubscribing from observables.
- Detaching event listeners.
- Cancelling ongoing HTTP requests.
- Clearing timers or intervals.

Using **ngOnDestroy()** helps prevent memory leaks by ensuring that resources are released when the component is no longer needed.

Example:

```
ngOnDestroy() {  
  if (this.subscription) {  
    this.subscription.unsubscribe();  
  }  
}
```

40. What are some common issues with routing in Angular and how can you resolve them?

Some common routing issues in Angular include:

1. 404 Page Not Found:

- Ensure that the routes are correctly defined in `app-routing.module.ts`.
- Check for typos in the route paths and that routes are imported correctly.

2. Missing or Incorrect Route Parameters:

- Verify that route parameters (e.g., `id`) are passed correctly, and use `ActivatedRoute` to retrieve them in the target component.

3. Lazy Loading Failures:

- Ensure that the module is correctly configured for lazy loading, and the **loadChildren** syntax is accurate.
- Check for errors in the module paths or missing dependencies.

4. Relative Navigation Issues:

- Ensure correct use of relative paths in router navigation.
- Use `relativeTo: this.route` when navigating relative to the current route.

5. Route Guard Failures:

- Ensure that route guards are implemented and configured properly to protect routes.

Experienced (Q&A)

1. Explain the concept of Angular's Ahead-of-Time (AOT) compilation.

Ahead-of-Time (AOT) compilation is a key feature of Angular that allows the application to be compiled during the build process rather than at runtime. This provides several benefits:

- **Faster Rendering:** AOT compiles the application at build time, generating efficient JavaScript code. This means that when the application is loaded in the browser, the browser doesn't need to compile the app on the fly, leading to faster load times.
- **Smaller Bundles:** Since Angular can pre-compile templates and components into efficient JavaScript code, the final output bundle is smaller compared to Just-in-Time (JIT) compilation.
- **Early Error Detection:** AOT compilation catches template errors and other issues at build time, which improves the development experience by reducing runtime errors.
- **Optimized Change Detection:** AOT performs better with change detection because the template is already compiled into highly optimized JavaScript code.

To enable AOT in Angular, you can build the application using the following command:

```
ng build --prod
```

This command triggers the AOT compilation along with other optimizations like minification and tree shaking.

2. What is Angular Universal, and why is it used?

Angular Universal is Angular's platform for server-side rendering (SSR) of Angular applications. It allows you to pre-render the HTML of an Angular application on the server, then send it to the client, making the application more SEO-friendly and improving initial load performance.

Benefits of Angular Universal:

- **Improved SEO:** Since the HTML is pre-rendered, search engines can easily index the content of your app, leading to better search rankings for content-heavy applications.
- **Faster Initial Rendering:** Server-side rendering can provide a faster initial page load because the user receives pre-rendered HTML, rather than waiting for JavaScript to load and execute in the browser.
- **Social Media Previews:** Since platforms like Facebook and Twitter crawl HTML content, SSR helps ensure that links shared from your Angular application show accurate metadata (like title, description, and image previews).

To implement Angular Universal, you can generate an SSR-enabled application using the Angular CLI:

```
ng add @nguniversal/express-engine
```

3. How can you improve the performance of an Angular application?

Improving the performance of an Angular application can be done using several strategies:

- **Lazy Loading:** Split your application into smaller feature modules that are loaded only when necessary, reducing the initial load time.
- **Ahead-of-Time (AOT) Compilation:** As discussed earlier, AOT reduces the amount of work that Angular does at runtime, leading to faster performance.
- **Change Detection Strategy:** Use `ChangeDetectionStrategy.OnPush` to limit change detection to only when the component's inputs change or an event triggers an update. This prevents Angular from checking every component on every change detection cycle.

TrackBy with ngFor: Use **trackBy** with **ngFor** to avoid unnecessary re-renders when the list changes.

```
<div *ngFor="let item of items; trackBy: trackById">
  {{ item.name }}
</div>
```

- **Service Workers:** Implement service workers to cache static assets and API responses. This improves performance, especially on subsequent visits, and supports offline functionality.
- **Tree Shaking:** Ensure tree-shaking is enabled (which is done by default in Angular CLI with production builds) to eliminate unused code from the final bundle.
- **Minification:** Minify JavaScript and CSS to reduce the size of the files sent over the network. This can be achieved by using the `--prod` flag during builds.
- **Bundle Optimization:** Reduce the size of the application by breaking it into smaller chunks using **lazy loading**, and by excluding unnecessary libraries.

4. What are the key differences between Reactive Forms and Template-Driven Forms in Angular?

Reactive Forms and **Template-Driven Forms** are two approaches to managing forms in Angular. Here are the key differences:

Feature

Reactive Forms

Template-Driven Forms

Approach

Code-centric (more control in TypeScript)

Template-centric (logic in the template)

Form Model

Synchronous model in the component

Asynchronous model in the template

Validation

Built-in and custom validators directly in the form model

Uses directives in templates like required

Form Control Initialization

Programmatically using FormControl, FormGroup

Automatically through template directives

Use Case

Complex forms with dynamic validation and interactions

Simple, static forms with less complex validation

Scaling

More scalable for larger forms

Better for simple forms

Example of Reactive Form:

```
this.form = this.fb.group({  
  name: ['', [Validators.required]],  
  email: ['', [Validators.required, Validators.email]]  
});
```

Example of Template-Driven Form:

```
<form #form="ngForm">  
  <input name="name" ngModel required />  
  <input name="email" ngModel required email />  
</form>
```

5. Explain how Angular's Dependency Injection works in depth.

Dependency Injection (DI) in Angular is a design pattern used to implement IoC (Inversion of Control). It allows Angular to manage the creation and injection of

dependencies into components, services, and other constructs.

Key Concepts:

- **Providers:** These are instructions that tell Angular how to create or retrieve an instance of a dependency.
- **Injectors:** These are responsible for creating and injecting dependencies at runtime.
- **Injection Tokens:** These are unique identifiers used to configure and access services.

When Angular creates a component, it injects all required dependencies (services, other components) through the constructor using the **@Injectable()** decorator.

Example of Dependency Injection in Angular:

```
@Injectable()
export class MyService {
  constructor() {}
}

@Component({
  selector: 'app-root',
  providers: [MyService]
})
export class AppComponent {
  constructor(private myService: MyService) {}
}
```

In this example, Angular's DI system will inject an instance of MyService into AppComponent automatically.

Dependency Injection Scopes:

- **Root-level:** Services provided in the @Injectable() decorator or AppModule are singleton across the application.
- **Component-level:** Providers scoped to a component's injector only, which means the service is created for that specific component and its children.

6. How would you configure lazy loading for feature modules in Angular?

Lazy loading in Angular allows you to load feature modules on demand instead of loading them upfront when the application starts. This improves the initial load time of your application.

Steps to configure lazy loading:

Create a feature module (if not already created):

```
ng generate module feature
```

Create a routing module for the feature:

```
ng generate module feature/feature-routing --flat --module=feature
```

2.

Define the routes in the feature module:

```
const routes: Routes = [  
  {  
    path: '',  
    component: FeatureComponent  
  }  
];  
  
@NgModule({  
  imports: [RouterModule.forChild(routes)],  
  exports: [RouterModule]  
})  
export class FeatureRoutingModule {}
```

Configure lazy loading in the main routing module (app-routing.module.ts):

```
const routes: Routes = [  
  {  
    path: 'feature',  
    loadChildren: () => import('./feature/feature.module').then(m =>  
      m.FeatureModule)  
  }  
];
```

```
    }  
  ];  
  
  @NgModule({  
    imports: [RouterModule.forRoot(routes)],  
    exports: [RouterModule]  
  })  
  export class AppRoutingModule {}
```

This configures the **feature module** to be lazily loaded when the `/feature` route is accessed.

7. What are the advantages of using Angular CLI in a large application?

The **Angular CLI** provides several advantages when working with large applications:

1. **Efficiency:** The CLI automates many tasks, such as generating components, services, modules, and setting up routing, which speeds up development.
2. **Consistency:** It ensures a consistent project structure and development process across teams.
3. **Build Optimizations:** The CLI provides commands to build the application for production (`ng build --prod`), which enables optimizations such as Ahead-of-Time (AOT) compilation, tree shaking, and minification.
4. **Testing Integration:** It integrates with tools like Karma, Jasmine, and Protractor to simplify unit testing and end-to-end testing.
5. **Code Generation:** Angular CLI can generate boilerplate code for components, services, and other features, helping developers focus on logic rather than setup.
6. **Built-in Development Server:** It provides a development server (`ng serve`) to serve the application with live reload functionality.
7. **Configuration Management:** The CLI helps manage environment-specific configurations with ease, making it easier to manage builds for multiple environments.

8. What are Angular decorators and how do they help in application development?

Decorators in Angular are special types of functions used to add metadata to classes, methods, and properties. They enable Angular to associate certain functionality with specific classes or methods.

Some key Angular decorators include:

@Component: Defines a component and its associated template and styles.

```
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {}
```

@Injectable: Marks a class as available for dependency injection.

```
@Injectable()
export class MyService {}
```

@NgModule: Defines a module and its dependencies.

```
@NgModule({
  declarations: [AppComponent],
  imports: [BrowserModule],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

1. **@Input and @Output:** Used for passing data between components.
2. **@HostListener and @HostBinding:** Handle DOM events and bind data to DOM properties.

Decorators help in organizing code, making it declarative and easy to maintain.

9. How does Angular handle memory leaks, and what are some strategies to avoid them?

Angular provides mechanisms to manage memory effectively, but developers must take care to avoid memory leaks, especially in larger applications.

Common Causes of Memory Leaks:

1. **Unsubscribed Observables:** If subscriptions to observables are not properly unsubscribed, it leads to memory leaks.
Solution: Use **ngOnDestroy** to unsubscribe or use the **async pipe**, which automatically handles subscriptions.
2. **Event Listeners:** If event listeners are not cleaned up, they can cause memory leaks.
Solution: Use **ngOnDestroy** to remove event listeners or rely on Angular's built-in **Renderer2** for safe DOM manipulation.
3. **Detached DOM Nodes:** Keeping references to DOM elements outside of Angular's change detection can cause memory leaks.
Solution: Always rely on Angular's built-in directives and change detection mechanisms to manage DOM elements.
4. **Service Leaks:** Services injected at the component level might hold unnecessary references if not disposed of correctly.
Solution: Use the **providedIn** syntax in services to ensure services are scoped correctly and do not hold unnecessary references.

10. Explain the use of `ngOnInit()` and `ngAfterViewInit()` lifecycle hooks with examples.

Both **ngOnInit** and **ngAfterViewInit** are lifecycle hooks that are executed during different stages of the component lifecycle.

- **ngOnInit:**
 - Called once when the component is initialized. It's ideal for component initialization tasks, such as fetching data or setting up initial values.

Example:

```
ngOnInit() {  
  console.log('Component Initialized');  
}
```

- **ngAfterViewInit:**
 - Called after the component's view (DOM elements) has been initialized. It's useful when you need to interact with the DOM elements or child components.

```
ngAfterViewInit() {
  console.log('Component View Initialized');
}
```

The primary difference is that **ngOnInit** is for initializing the component's input properties, whereas **ngAfterViewInit** is for working with the component's view after the view's DOM elements are available.

11. How do you secure an Angular application from cross-site scripting (XSS) attacks?

Cross-Site Scripting (XSS) attacks occur when malicious scripts are injected into a web page, which can then be executed in the browser of other users. In Angular, several mechanisms and best practices are used to prevent XSS:

Automatic HTML Escaping: Angular automatically escapes content interpolated inside `{{ }}` syntax. This ensures that user input or dynamic content is rendered as plain text rather than HTML or JavaScript, preventing any embedded scripts from being executed.

```
<div>{{ userInput }}</div>
```

1. Even if `userInput` contains malicious JavaScript, Angular escapes it, ensuring it's treated as text.

Sanitization: Angular's **DomSanitizer** service helps sanitize dynamic content, such as HTML, URLs, styles, etc. For example, when rendering dynamic HTML or a URL, you can use `DomSanitizer` to ensure it's safe to use.

```
import { DomSanitizer } from '@angular/platform-browser';

constructor(private sanitizer: DomSanitizer) {}

getSafeHtml(dangerousHtml: string) {
  return this.sanitizer.bypassSecurityTrustHtml(dangerousHtml);
}
```

}

1. **Avoid Dangerous Methods:** Avoid directly using `innerHTML` or `outerHTML`, as these methods do not automatically sanitize input and can lead to XSS vulnerabilities. If you need to dynamically set HTML, always use the `DomSanitizer`.
2. **Strict Input Validation:** Ensure that inputs are properly validated on both the client and server sides. This can be done by checking user input against allowed patterns or using regular expressions to filter out potentially dangerous characters.
3. **Content Security Policy (CSP):** Implementing CSP headers (like `Content-Security-Policy`) helps protect your application from XSS by restricting where scripts can be loaded from and ensuring that only trusted sources are used.
4. **Secure Template Syntax:** In Angular, avoid using `dangerouslySetInnerHTML` or any raw JavaScript execution methods. Always prefer binding with Angular's syntax (`{{}}`) and `ngIf` or `ngFor` for DOM manipulation.

Check out these other Interview Questions...

Interviews, tips, guides, industry best practices, and news.

[Swift Interview Questions and Answers](#)

[Cobol Interview Questions and Answers](#)

[Java Debugging Interview Questions and Answers](#)

[Qlik Sense Interview Questions and Answers](#)

[Ruby on Rails Interview Questions and Answers](#)

[ELT interview Questions and Answers](#)

[Operating System
Interview Questions and
Answers](#)

[AutoCAD Interview
Questions and Answers](#)

[Data Science interview
Questions and Answers](#)

[View all posts](#)

 WeCP is the #1 AI Talent Assessment Platform to assess, interview, hire, and upskill candidates who truly deserve to work with you. Copyright © 2026 WeCP (We Create Problems). All rights reserved. Backed By 	Platform	Company	Compare	Resources	Solutions
	AI Interviews	About Us	WeCP Vs Hirevue	Blogs	Launch AI Interviewers
	AI Assessments	Our Customers	WeCP Vs HackerRank	Email Templates	Evaluate AI-ready Technical Talent
	Panel Interviews	Wall Of Love 	WeCP Vs Coderpad	Job Descriptions	Measure Communication and Soft Skills
	WeCP AI	Newsroom	WeCP Vs TestGorilla	Interview Questions	Run Human Interviews with an AI Companion
	English Pro	Careers	WeCP Vs Codility	Case Study	
	Culture Pro	Contact	View All →	Voice Integrations	
				Change Log	
				Test Library	
				AI-Powered Quiz Maker	
				AI-Powered Question Generator	
				Online Hackathon Platform	
				Online Proctoring Software	

SF, USA	AUSTIN, USA	BENGALURU, INDIA	PUNE, INDIA	LONDON, UK
3388 17th St San Francisco, California – 94110 USA	701 Tillery St, Austin, Texas – 78702 USA	2728, 27th Main Road, Sector 1, HSR Layout, Bangalore – 560102 India	WeCP, Wework Raheja Woods, Kalyani Nagar, Pune – 411006 India	PO Box 1487, Peterborough, PE1 9XX, UK